AGM RURAL COLLEGE OF ENGINEERING AND TECHNOLOGY, VARUR

Department of Computer Science & Engineering

Web Technology Laboratory

Laboratory Manual





BCSL504

Prepared by, Mr. Chandrakant Hatti

Index

Sl. No	Experiment Name	Page No
i.	Course Details	ii
ii.	Basics of HTML	1
	Following Experiments to be done Using Notepad/Text Editor and Web Browser	
1.	HTML page to demonstrate basic HTML tags.	3
2.	HTML page to display your class time table.	5
3.	External CSS style sheet to apply for HTML tags.	9
4.	Develop HTML page to create registration form.	12
5.	Develop HTML page to create a newspaper.	16
6.	Apply HTML, CSS and JavaScript to design a simple calculator.	21
7.	Develop JavaScript program to work on JSON & CSV.	26
8.	(a) Develop a PHP program to display visitor count. (b) Develop a PHP program to sort data stored in database.	32
9.	Develop jQuery script.	35
10	Develop JavaScript program with AJAX.	40
	Appendix – I: HTML Tag List by Category	45
	Appendix – II: HTML Attributes	52
	Appendix – III: HTML Events	65

Course Details

Course Title/Code: Web Technology Laboratory (BCSL504)	*CIE Marks: 50*
Lecture Hours/Week: 02 Hours Laboratory	*SEE Marks: 50*
RBT Level: L1, L2, L3	*Exam Hours: 03*

Course Outcomes: At the end of the course the student will be able to:

1. Design the experiment for the given problem using HTML, Javascript and CSS.
2. Develop the solution for the given real-world problem using jQuery, Ajax and PHP.
3. Analyze the results and produce substantial written documentation.

Conduct of Practical Examination:

1. All laboratory experiments are to be included for practical examination.
2. Strictly follow the instructions as printed on the cover page of answer script for breakup of marks.
3. Change of experiment is allowed only once and Marks allotted to the procedure part to be made zero.

Web links (e-Resources):

- https://www.w3schools.com/html/default.asp
- https://www.w3schools.com/css/default.asp
- https://www.w3schools.com/js/js_examples.asp
- https://www.geeksforgeeks.org/javascript-examples/
- https://www.w3schools.com/php/default.asp
- https://www.w3schools.com/jquery/default.asp
- https://www.w3schools.com/js/js_ajax_intro.asp
- https://www.geeksforgeeks.org/jquery-tutorial/

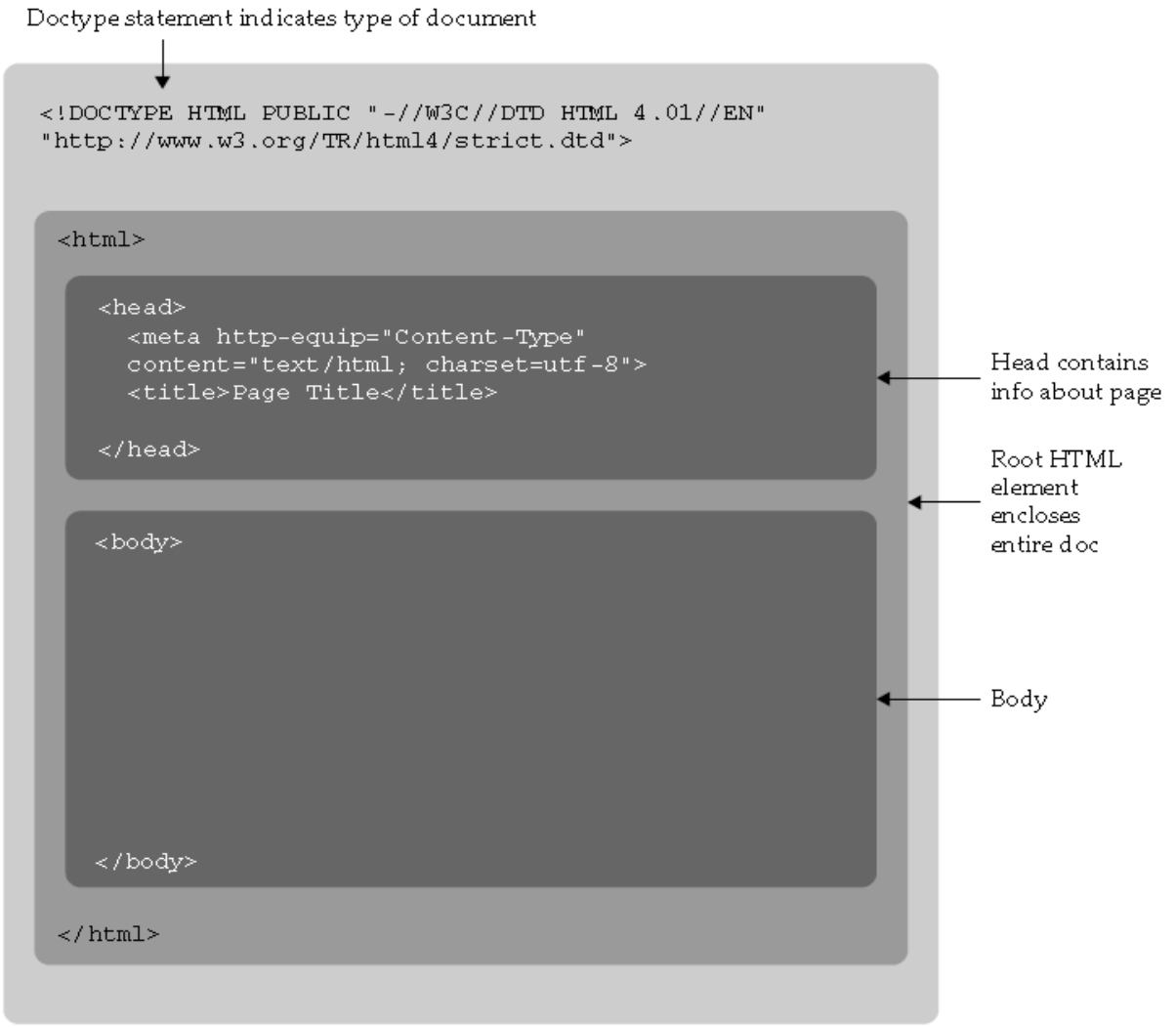
Reference Book:

1. Randy Connolly and Ricardo Hoar, Fundamentals of Web Development, 3rd edition, Pearson, 2021
2. Robert W Sebesta, Programming the World Wide Web, 8th Edition, Pearson Education, 2020.

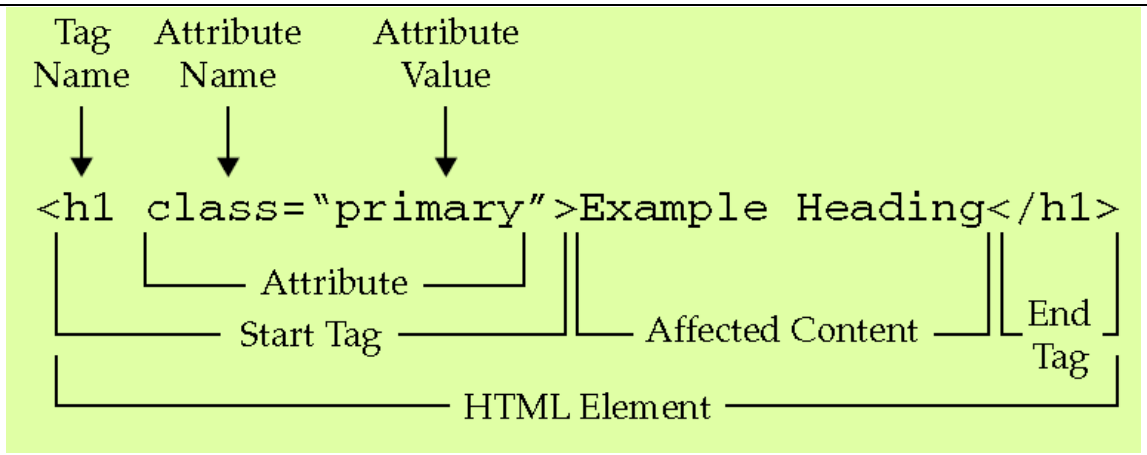
PREPARED BY
Mr. Chandrakant Hatti

Basics of HTML

1) HTML Structure



2) Syntax of HTML Element (tag)



Start Tag is the first part of an element. It contains *Tag Name* and *Attribute.*
Affected Content is the text to be displayed on the screen.
End Tag is the end or closure of an element by adding *slash (/)* within it.

3) Procedure to Execute the HTML Program

Follow these steps to develop and execute the HTML page.

Step 1: Create the HTML File

- Open a text editor (e.g., Notepad, Sublime Text, VS Code).
- Create a new file and save it as FileName.html (FileName means name of the file)

Step 2: Write the HTML Code

- Type the code into the saved file as per the aim of the program.

Step 3: Save the HTML File

- After writing the code, save the file.

Step 4: Open the HTML File in a Web Browser

- Locate the saved file on your computer.
- Double click to Open the file in the default browser OR Right-click the file and choose "Open with" your preferred web browser (e.g., Chrome, Firefox, Edge, etc).

Step 5: Review and Test

- Ensure that all tags and styles are displayed as expected in the browser.
- If any changes are needed, reopen the file in your text editor, make the necessary adjustments, and save the file again.
- Refresh the browser to view the updated content.

Experiment – 1

Develop the HTML page named as "Myfirstwebpage.html". Add the following tags with relevant content.

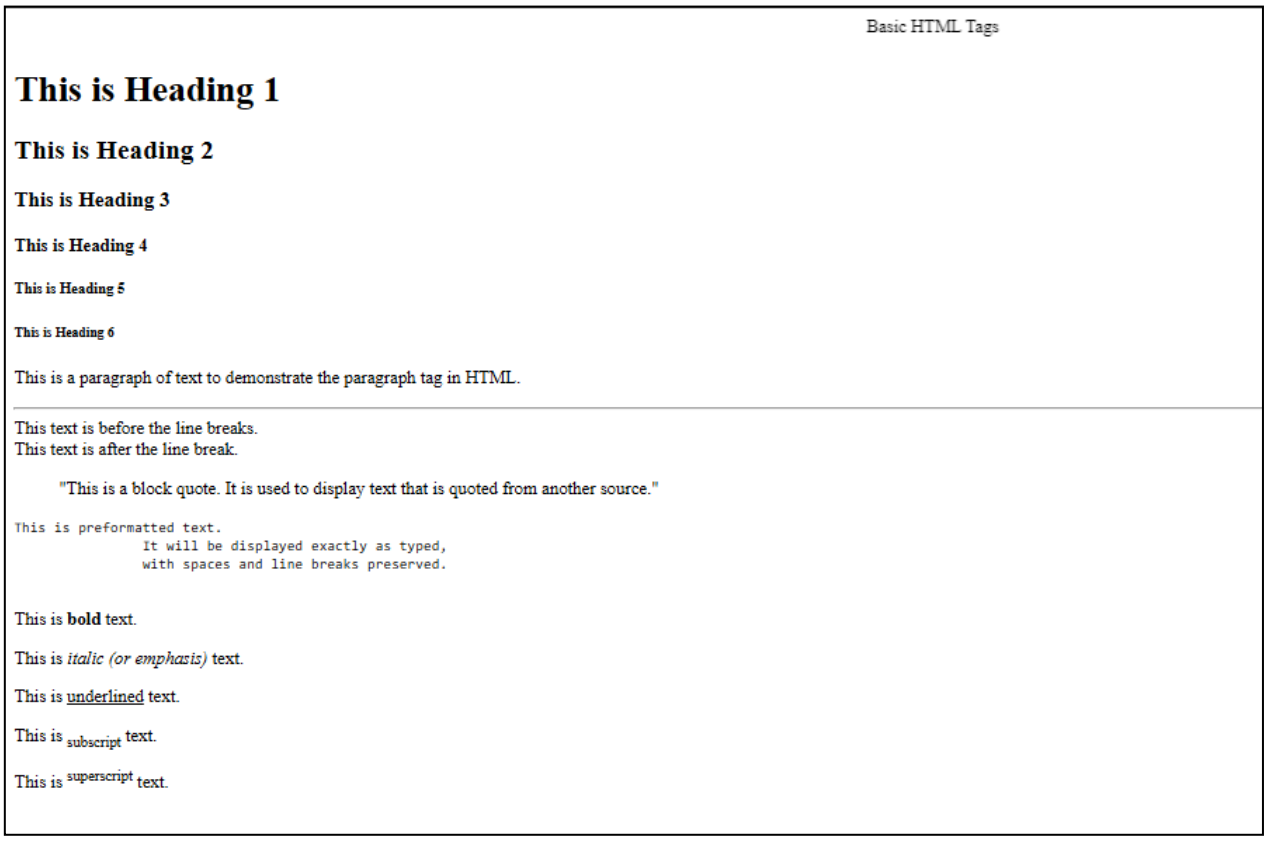
1. **Set the title of the page as "My First Web Page"**
2. **Within the body use the following tags:**
    a) **Moving text "Basic HTML Tags"**
    b) **Different heading tags (h1 to h6)**
    c) **Paragraph**
    d) **Horizontal line**
    e) **Line Break**
    f) **Block Quote**
    g) **Pre tag**
    h) **Different Logical Style (<b>, <u>, <sub>, <sup> etc.)**

HTML Program:

```
<!DOCTYPE html>
<html>
  <head>
     <title>My First Web Page</title>
  </head>
  <body>
     <! -- Moving Text -->
     <marquee>Basic HTML Tags</marquee>
     <! -- Heading Tags -->
     <h1>This is Heading 1</h1>
     <h2>This is Heading 2</h2>
     <h3>This is Heading 3</h3>
     <h4>This is Heading 4</h4>
     <h5>This is Heading 5</h5>
     <h6>This is Heading 6</h6>
     <! -- Paragraph Tag -->
     <p>This is a paragraph of text to demonstrate the paragraph tag in HTML.
</p>
     <! -- Horizontal Line -->
     <hr>
     <! -- Line Break -->
     This text is before the line breaks.<br>This text is after the line break.
```

```
<! -- Block Quote -->
<blockquote>
"This is a block quote. It is used to display text that is quoted from another
  source."
</blockquote>
<! -- Preformatted Text -->
<pre>This is preformatted text.
It will be displayed exactly as typed,
with spaces and line breaks preserved.
</pre>
<! -- Logical Styles -->
<p>This is <b>bold</b> text. </p>
<p>This is <em>italic (or emphasis)</em> text. </p>
<p>This is <u>underlined</u> text. </p>
<p>This is <sub>subscript</sub> text. </p>
<p>This is <sup>superscript</sup> text. </p>
</body>
</html>
```

OUTPUT:



Experiment – 2

Develop the HTML page named "Table.html" to display your class timetable.

a) **Provide the title as "Time Table" with table header and table footer, row-span and col-span etc.**
b) **Provide various color options to the cells (Highlight the lab hours and elective hours with different colors.)**
c) **Provide color options for rows.**

Styles:

- The header (<thead>) and footer (<tfoot>) rows are styled with a green background and white text.
- Row-span and col-span are used to merge cells across rows and columns.
- The lab hours are highlighted with a yellow background, and elective hours are highlighted with an orange background.
- The rows are alternately colored to enhance readability.

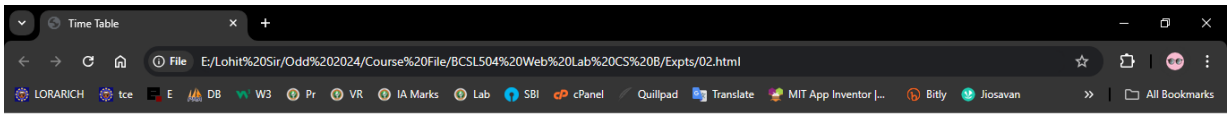
HTML Program:

```
<!DOCTYPE html>
<html>
  <head>
     <title>Time Table</title>
     <style>
     table, th, td {
          border:1px solid black;
          border-collapse:collapse;
          text-align:center;
     }
     thead,tfoot {
          background-color:green;
          color:white;
     }
     .lab {
          background-color:yellow;
     }
     .elective {
          background-color:orange;
     }
     </style>
```

```
</head>
<body>
    <h1>Class Time Table</h1>
    <!--Create Table -->
    <table style="width:100%;">
    <!-- Table Header -->
    <thead>
    <tr>
        <th>Day</th>
        <th>9:15 - 10:15</th>
        <th>10:15 - 11:15</th>
        <th>11:15 - 11:30</th>
        <th>11:30 - 12:30</th>
        <th>12:30 - 1:30</th>
        <th>1:30 - 2:30</th>
        <th>2:30 - 3:30</th>
        <th>3:30 - 4:30</th>
        <th>4:30 – 5:15</th>
    </tr>
    </thead>
    <!-- Table Body -->
    <tbody>
    <tr>
        <td><b>MON</b></td>
        <td>Math</td>
        <td>Science</td>
        <td rowspan="5" style="background-color: grey">Short Break</td>
        <td colspan="2" class="lab">Physics Lab</td>
        <td rowspan="5" style="background-color: grey">Lunch Break</td>
        <td>English</td>
        <td class="elective">Art Elective</td>
        <td>History</td>
    </tr>
    <tr>
        <td><b>TUE</b></td>
        <td>Math</td>
        <td>Science</td>
        <td>English</td>
        <td class="elective">Music Elective</td>
        <td>History</td>
        <td colspan="2">History</td>
    </tr>
```

```
    <tr>
        <td><b>WED</b></td>
        <td>Math</td>
        <td>Science</td>
        <td colspan="2">Computer Science</td>
        <td class="lab">Chemistry Lab</td>
        <td colspan="2">History</td>
    </tr>
    <tr>
        <td><b>THU</b></td>
        <td>Math</td>
        <td>Science</td>
        <td colspan="2">Computer Science</td>
        <td>English</td>
        <td class="elective">Drama Elective</td>
        <td>History</td>
    </tr>
    <tr>
        <td><b>FRI</b></td>
        <td>Math</td>
        <td>Science</td>
        <td colspan="2">Computer Science</td>
        <td>English</td>
        <td class="elective">Photography Elective</td>
        <td>History</td>
    </tr>
    </tbody>
    <!-- Table Footer -->
    <tfoot>
    <tr>
        <td colspan="10">End of Time Table</td>
    </tr>
    </tfoot>
    </table>
  </body>
</html>
```

OUTPUT:



Class Time Table

Day	9:15 - 10:15	10:15 - 11:15	11:15 - 11:30	11:30 - 12:30	12:30 - 1:30	1:30 - 2:30	2:30 - 3:30	3:30 - 4:30	4:30 – 5:15
MON	Math	Science	Short Break	Physics Lab		Lunch Break	English	Art Elective	History
TUE	Math	Science		English	Music Elective		History	History	
WED	Math	Science		Computer Science			Chemistry Lab	History	
THU	Math	Science		Computer Science			English	Drama Elective	History
FRI	Math	Science		Computer Science			English	Photography Elective	History
End of Time Table									



Experiment – 3

Develop an external style sheet named as "style.css" and provide different styles for h2, h3, hr, p, div, span, time, img & a tags. Apply different CSS selectors for tags and demonstrate the significance of each.

SOLUTION:

Follow these steps to develop and execute an HTML program with an external style sheet named style.

Step 1: Create the External CSS File

In the same directory where index.html is saved, create a new file and save it as style.css.

Step 2: Write the CSS Code & Save it.

Type the code into the style.css file and save it.

Step 3: Understanding CSS Selectors Used

Element Selector: Targets specific HTML tags (e.g., h2, h3, p) and applies the defined styles to all elements of that type.

Descendant Selector: Targets an element that is nested inside another element (e.g., span inside a div).

Pseudo-Class Selector: Targets elements in a specific state (e.g., a:hover to style links when the mouse hovers over them).

CSS File Program (style.css)

```
/* Styling for <h2> tag */
h2 {
    color: #2c3e50;
    font-size: 28px;
    text-align: center;
}

h3 {
    color: #e74c3c;
    font-size: 24px;
    text-align: left;
    margin-left: 20px;
}

hr {
    border: 1px solid #7f8c8d;
    margin: 20px 0;
}
```

```
p {
    color: #34495e;
    font-size: 16px;
    line-height: 1.6;
    text-align: justify;
    margin: 20px;
}
div {
    background-color: #f0f0f0;
    padding: 15px;
    margin: 20px;
    border: 1px solid #bdc3c7;
}
div span {
    color: #8e44ad;
    font-weight: bold;
}

time {
    color: #27ae60;
    font-size: 18px;
    display: block;
    margin: 20px;
}

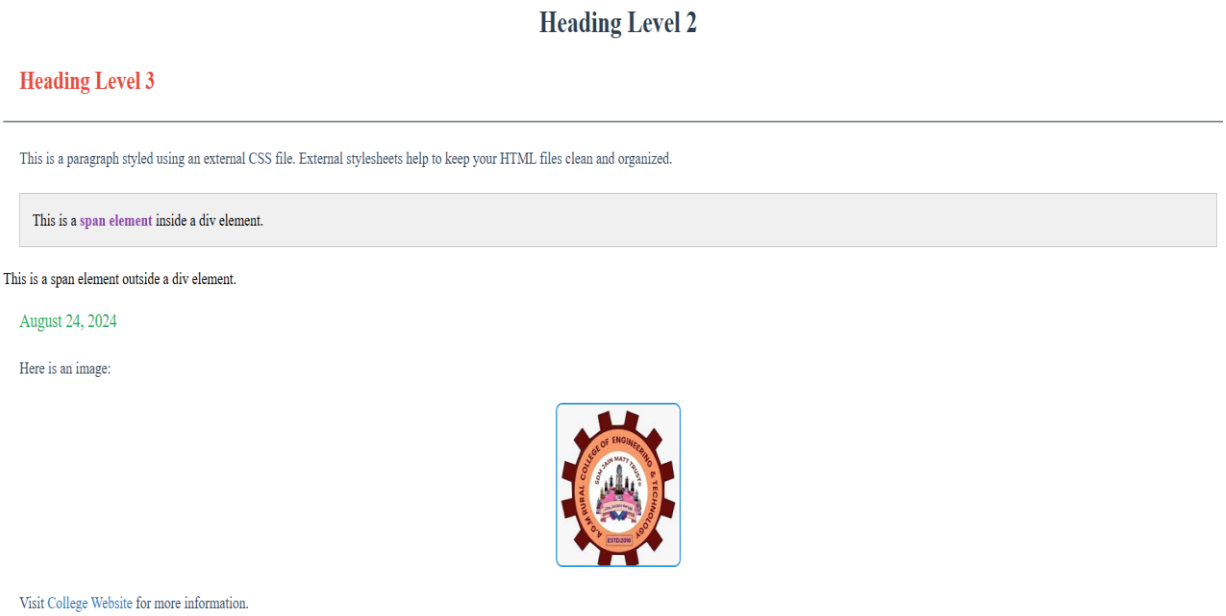
img {
    border: 2px solid #3498db;
    display: block;
    margin: 20px auto;
    border-radius: 8px;
    width:150px;
    height:150px;
}

a {
    color: #2980b9;
    text-decoration: none;
}
a:hover {
    color: #e67e22;
    text-decoration: underline;
}
```

HTML Program (index.html)

```
<!DOCTYPE html>
<html>
    <head>
        <title>External CSS Example</title>
        <link rel="stylesheet" href="style.css">
    </head>
    <body>
        <h2>Heading Level 2</h2>
        <h3>Heading Level 3</h3>
        <hr>
        <p>This is a paragraph styled using an external CSS file. External
            stylesheets help to keep your HTML files clean and organized.</p>
        <div>
            This is a <span>span element</span> inside a div element.
        </div>
        This is a <span>span element</span> outside a div element.
        <time datetime="2024-08-24">August 24, 2024</time>
        <p>Here is an image:</p>
        <img src="https://via.placeholder.com/150" alt="Sample Image">
        <p>Visit <a href="https://tce.ac.in" target="_blank">College Website</a>
            for more information.</p>
    </body>
</html>
```

OUTPUT:



Experiment – 4

Develop HTML page named as "registration.html" having variety of HTML input elements with background colors, table for alignment & provide font colors & size using CSS styles.

HTML Program

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <title>Registration Form</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">

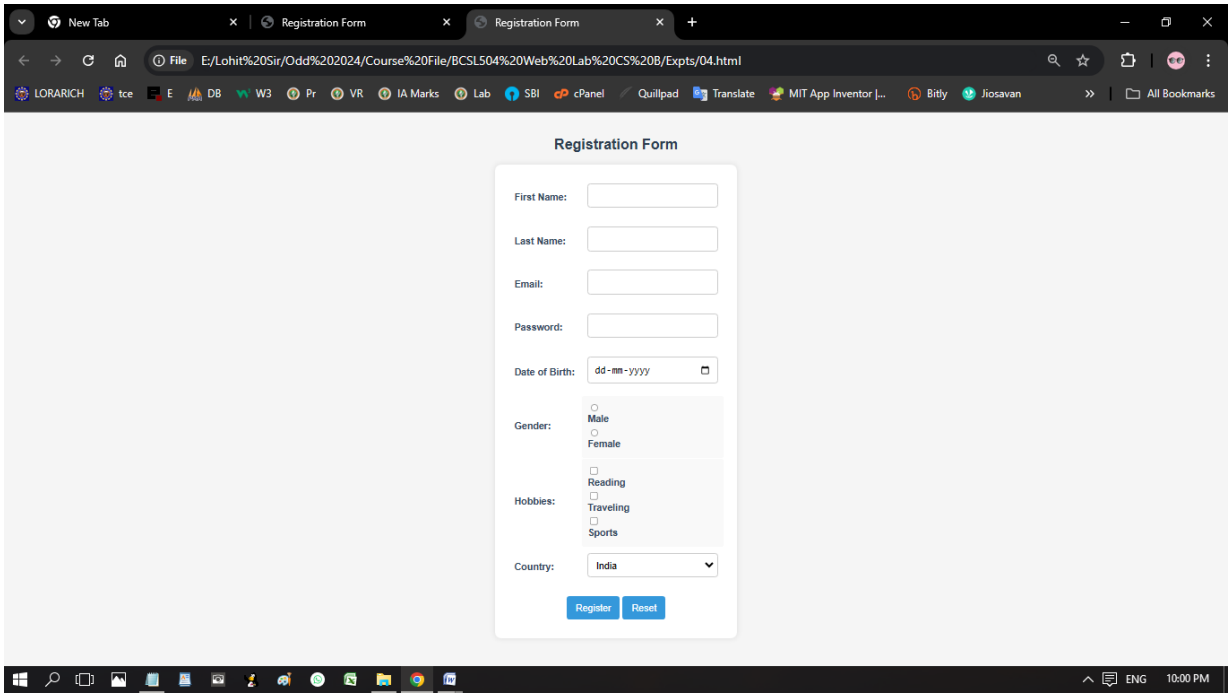
        <style>
        /* Styling the body */
        body {
            font-family: Arial, sans-serif;
            background-color: #f5f5f5;
            margin: 0;
            padding: 20px;
        }
        /* Styling the table */
        table {
            margin: 0 auto;
            padding: 20px;
            background-color: #ffffff;
            border-radius: 10px;
            box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
        }
        td {
            padding: 10px;
            font-size: 16px;
            color: #34495e;
        }
        /* Styling input elements */
        input[type="text"], input[type="email"], input[type="password"],
        input[type="date"], select {
            width: 100%;
            padding: 10px;
            border: 1px solid #ccc;
            border-radius: 5px;
            box-sizing: border-box;
```

```
            font-size: 16px;
            margin-bottom: 10px;
        }
        /* Styling buttons */
        input[type="submit"], input[type="reset"] {
            background-color: #3498db;
            color: white;
            padding: 10px 15px;
            border: none;
            border-radius: 5px;
            cursor: pointer;
            font-size: 16px;
        }
        input[type="submit"]:hover, input[type="reset"]:hover {
            background-color: #2980b9;
        }
        /* Styling the form header */
        h2 {
            text-align: center;
            color: #2c3e50;
            margin-bottom: 20px;
        }
        /* Specific input background colors */
        .gender, .hobbies {
            background-color: #f9f9f9;
            padding: 10px;
            border-radius: 5px;
        }
        /* Styling labels */
        label {
            font-weight: bold;
            display: block;
            margin-bottom: 5px;
            color: #34495e;
        }
    </style>
</head>
<body>
    <h2>Registration Form</h2>
    <form>
        <table>
        <tr>
```

```
    <td><label for="fname">First Name:</label></td>
    <td><input type="text" id="fname" name="firstname" required></td>
</tr>
<tr>
    <td><label for="lname">Last Name:</label></td>
    <td><input type="text" id="lname" name="lastname" required></td>
</tr>
<tr>
    <td><label for="email">Email:</label></td>
    <td><input type="email" id="email" name="email" required></td>
</tr>
<tr>
    <td><label for="password">Password:</label></td>
    <td><input type="password" id="password" name="password"
          required></td>
</tr>
<tr>
    <td><label for="dob">Date of Birth:</label></td>
    <td><input type="date" id="dob" name="dob" required></td>
</tr>
<tr>
    <td><label>Gender:</label></td>
    <td class="gender">
        <input type="radio" id="male" name="gender" value="male">
        <label for="male">Male</label>
        <input type="radio" id="female" name="gender" value="female">
        <label for="female">Female</label>
    </td>
</tr>
<tr>
    <td><label>Hobbies:</label></td>
    <td class="hobbies">
    <input type="checkbox" id="reading" name="hobbies" value="reading">
    <label for="reading">Reading</label>
    <input type="checkbox" id="traveling" name="hobbies"
    value="traveling">
    <label for="traveling">Traveling</label>
    <input type="checkbox" id="sports" name="hobbies" value="sports">
    <label for="sports">Sports</label>
    </td>
</tr>
<tr>
```

```
            <td><label for="country">Country:</label></td>
            <td>
                <select id="country" name="country">
                <option value="ind">India</option>
                <option value="canada">Canada</option>
                <option value="uk">United Kingdom</option>
                </select>
            </td>
        </tr>
        <tr>
            <td colspan="2" style="text-align: center;">
                <input type="submit" value="Register">
                <input type="reset" value="Reset">
            </td>
        </tr>
        </table>
    </form>
</body>
</html>
```

OUTPUT:



Experiment – 5

Develop HTML page named as "newpaper.html" having variety of HTML semantic elements with background colors, text-colors & size for figure, table, aside, section, article, header, footer… etc.

HTML Program:

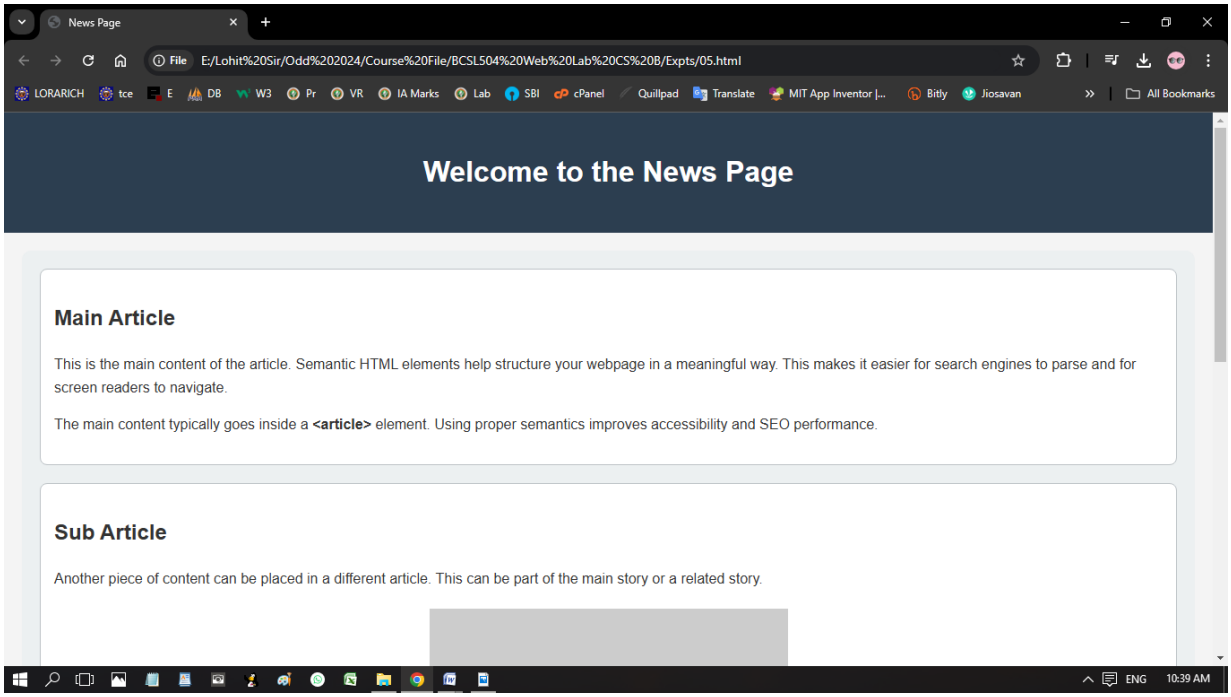
```
<!DOCTYPE html>
<html>
    <head>
        <title>News Page</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <style>
        /* General Page Styles */
        body {
            font-family: Arial, sans-serif;
            line-height: 1.6;
            background-color: #f4f4f4;
            color: #333;
            margin: 0;
            padding: 0;
        }
        header, footer {
            background-color: #2c3e50;
            color: white;
            text-align: center;
            padding: 20px 0;
        }
        /* Main article section styling */
        section {
            background-color: #ecf0f1;
            margin: 20px;
            padding: 20px;
            border-radius: 10px;
        }
        article {
            background-color: #ffffff;
            padding: 15px;
            margin-bottom: 20px;
            border: 1px solid #bdc3c7;
            border-radius: 8px;
        }
```

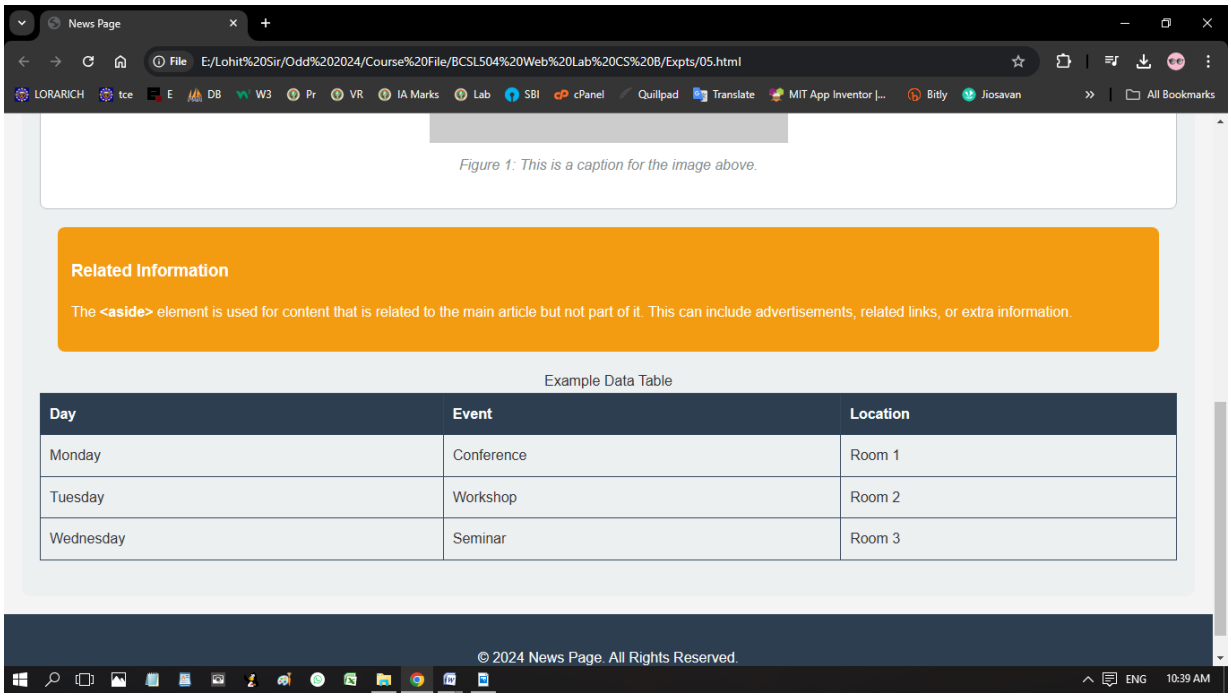
```
        /* Aside styling */
        aside {
            background-color: #f39c12;
            color: white;
            padding: 15px;
            margin: 20px;
            border-radius: 8px;
        }
        /* Figure and figcaption styling */
        figure {
            margin: 20px;
            text-align: center;
        }
        figcaption {
            font-style: italic;
            color: #7f8c8d;
            margin-top: 5px;
        }
        /* Table styling */
        table {
            width: 100%;
            border-collapse: collapse;
            margin: 20px 0;
        }
        th, td {
            border: 1px solid #34495e;
            padding: 10px;
            text-align: left;
        }
        th {
            background-color: #2c3e50;
            color: white;
        }
        /* Footer styles */
        footer {
            margin-top: 20px;
        }
        </style>
    </head>
```

```
<body>
    <header>
        <h1>Welcome to the News Page</h1>
    </header>
    <section>
        <article>
            <h2>Main Article</h2>
                <p>This is the main content of the article. Semantic HTML
                elements help structure your webpage in a meaningful way.
                This makes it easier for search engines to parse and for screen
                readers to navigate.</p>
                <p>The main content typically goes inside a
                <strong>&lt;article&gt;</strong> element. Using proper
                semantics improves accessibility and SEO performance.</p>
        </article>
        <article>
            <h2>Sub Article</h2>
                <p>Another piece of content can be placed in a different article.
                This can be part of the main story or a related story.</p>
            <figure>
                    <img src="https://via.placeholder.com/400" alt="Sample
                    Image" width="400" height="200">
                     <figcaption>Figure 1: This is a caption for the image
                     above.</figcaption>
            </figure>
        </article>
        <aside>
            <h3>Related Information</h3>
                <p>The <strong>&lt;aside&gt;</strong> element is used for
                content that is related to the main article but not part of it. This
                can include advertisements, related links, or extra
                information.</p>
        </aside>
        <table>
        <caption>Example Data Table</caption>
        <thead>
        <tr>
            <th>Day</th>
            <th>Event</th>
            <th>Location</th>
        </tr>
        </thead>
```

```
            <tbody>
            <tr>
                <td>Monday</td>
                <td>Conference</td>
                <td>Room 1</td>
            </tr>
            <tr>
                <td>Tuesday</td>
                <td>Workshop</td>
                <td>Room 2</td>
            </tr>
            <tr>
                <td>Wednesday</td>
                <td>Seminar</td>
                <td>Room 3</td>
            </tr>
            </tbody>
            </table>
        </section>
        <footer>
            <p>&copy; 2024 News Page. All Rights Reserved.</p>
        </footer>
    </body>
</html>
```

OUTPUT:




News Page
File E:/Lohit%20Sir/Odd%202024/Course%20File/BCSL504%20Web%20Lab%20CS%20B/Expts/05.html
LORARICH tce E DB W3 Pr VR IA Marks Lab SBI cPanel Quillpad Translate MIT App Inventor |... Bitly Jiosavan All Bookmarks
Figure 1: This is a caption for the image above.
Related Information
The <aside> element is used for content that is related to the main article but not part of it. This can include advertisements, related links, or extra information.
Example Data Table
Day Event Location
Monday Conference Room 1
Tuesday Workshop Room 2
Wednesday Seminar Room 3
© 2024 News Page. All Rights Reserved.
ENG 10:39 AM

Experiment – 6

Apply HTML, CSS and JavaScript to design a simple calculator to perform the following operations: sum, product, difference, remainder, quotient, power, square-root and square.

HTML Program

```
<!DOCTYPE html>
<html lang="en">
  <head>
  <title>Simple Calculator</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <style>
      body {
          font-family: Arial, sans-serif;
          background-color: #f0f0f0;
          color: #333;
          text-align: center;
          margin: 0;
          padding: 0;
      }
      h1 {
          margin-top: 50px;
          color: #2c3e50;
      }
      .calculator {
          background-color: #ffffff;
          padding: 20px;
          border-radius: 10px;
          box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
          display: inline-block;
          margin-top: 30px;
      }
      input[type="number"] {
          width: 100%;
          padding: 10px;
          margin: 10px 0;
          border: 1px solid #ccc;
          border-radius: 5px;
          font-size: 18px;
      }
```

```
        .buttons {
            margin: 10px 0;
        }
        button {
            background-color: #3498db;
            color: white;
            padding: 10px 20px;
            border: none;
            border-radius: 5px;
            margin: 5px;
            font-size: 16px;
            cursor: pointer;
        }
        button:hover {
            background-color: #2980b9;
        }
        .result {
            margin-top: 20px;
            font-size: 20px;
            font-weight: bold;
            color: #e74c3c;
        }
    </style>
    </head>
    <body>
        <h1>Simple Calculator</h1>
        <div class="calculator">
            <input type="number" id="num1" placeholder="Enter first number">
            <input type="number" id="num2" placeholder="Enter second number">
            <div class="buttons">
                <button onclick="performOperation('sum')">Sum</button>
                <button onclick="performOperation('product')">Product</button>
                <button
                onclick="performOperation('difference')">Difference</button>
                <button
                onclick="performOperation('remainder')">Remainder</button>
                <button onclick="performOperation('quotient')">Quotient</button>
                <button onclick="performOperation('power')">Power</button>
                <button onclick="performOperation('squareRoot')">Square
                Root</button>
                <button onclick="performOperation('square')">Square</button>
            </div>
```

```
        <div class="result" id="result"></div>
    </div>
    <script>
    function performOperation(operation) {
        let num1 = parseFloat(document.getElementById('num1').value);
        let num2 = parseFloat(document.getElementById('num2').value);
        let result;
        switch (operation) {
            case 'sum':
                result = num1+" + "+num2+" = "+(num1 + num2); break;
            case 'product':
                result = num1+" x "+num2+" = "+(num1 * num2); break;
            case 'difference':
                result = num1+" - "+num2+" = "+(num1 - num2); break;
            case 'remainder':
                result = num1+" % "+num2+" = "+(num1 % num2); break;
            case 'quotient':
                result = num1+" / "+num2+" = "+Math.floor(num1 / num2);
                break;
            case 'power':
                result = num1+" ^ "+num2+" = "+Math.pow(num1, num2);
                break;
            case 'squareRoot':
                 result = "Squareroot of "+num1+" is
                 "+Math.sqrt(num1)+"<br/>"+"Squareroot of "+num2+" is
                 "+Math.sqrt(num2); break;
            case 'square':
                result = "Square of "+num1+" is "+Math.pow(num1,
                2)+"<br/>"+"Square of "+num2+" is "+Math.pow(num2, 2);
                break;
            default:
                result = 'Invalid Operation';
        }
        document.getElementById('result').innerHTML = 'Result: ' + result;
    }
    </script>
  </body>
</html>
```

Explanation

1. HTML Structure

Input Fields: There are two input fields where users can enter numbers. Both input fields accept numbers (type="number").

Buttons: Each operation has a corresponding button, such as Sum, Product, Difference, etc. When clicked, they call the performOperation() function in JavaScript.

Result Section: The result of the calculation is displayed in a with the class result and id="result".

2. CSS Styling

Page Layout: The body has centered content with padding. A simple calculator box is created with padding, a border-radius, and a box-shadow to make it visually appealing.

Buttons: The buttons are styled with a blue background, white text, and rounded corners. A hover effect is added to change the background color when the mouse hovers over a button.

Result Display: The result is displayed in a bold, red font, giving it prominence on the page.

3. JavaScript Logic

The performOperation() function handles all the operations:

Sum: Adds the two input numbers.

Product: Multiplies the two input numbers.

Difference: Subtracts the second input number from the first.

Remainder: Calculates the remainder of the division of the first number by the second.

Quotient: Divides the first number by the second.

Power: Raises the first number to the power of the second number using Math.pow().

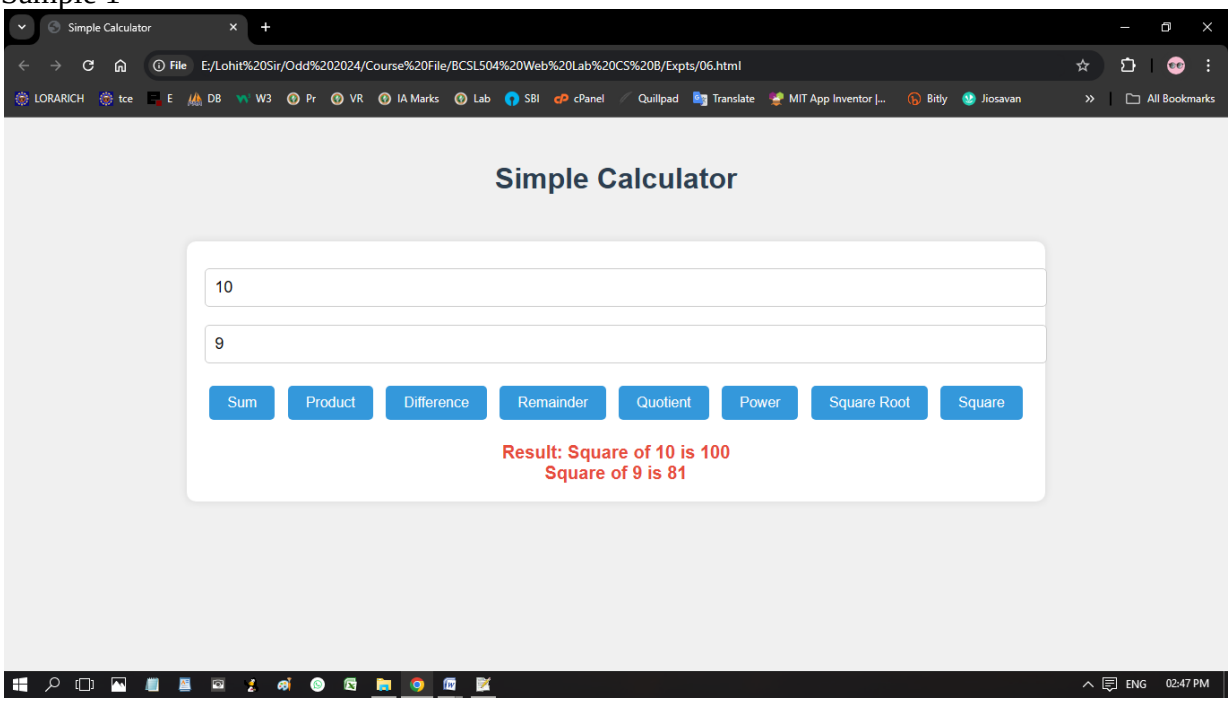
Square Root: Calculates the square root of the both numbers using Math.sqrt().

Square: Squares both numbers using Math.pow(num1, 2).

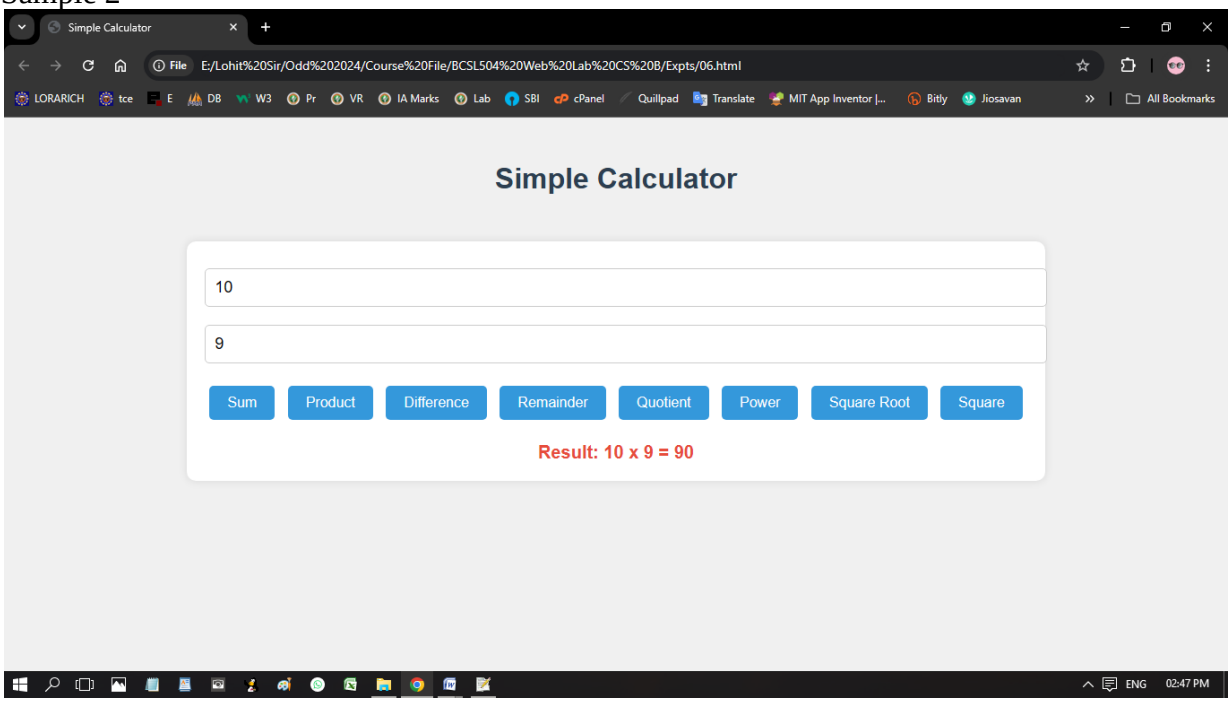
After performing the selected operation, the result is displayed in the result div.

OUTPUT:

Sample 1



Sample 2



Experiment – 7

Develop JavaScript program (with HTML/CSS) for:

a) **Converting JSON text to JavaScript Object**
b) **Convert JSON results into a date**
c) **Converting From JSON To CSV and CSV to JSON**
d) **Create hash from string using crypto.createHash() method.**

JavaScript with HTML/CSS Program:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>JSON/CSV Converter and Hash Generator</title>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <script src="https://cdnjs.cloudflare.com/ajax/libs/crypto-js/4.1.1/crypto-
     js.min.js"></script>
    <style>
      body {
          font-family: Arial, sans-serif;
          line-height: 1.6;
          margin: 0;
          padding: 20px;
          background-color: #f4f4f4;
      }
      .container {
          max-width: 800px;
          margin: auto;
          background: white;
          padding: 20px;
          border-radius: 5px;
          box-shadow: 0 0 10px rgba(0,0,0,0.1);
      }
      h1 {
          color: #333;
      }
      textarea {
          width: 100%;
          height: 100px;
          margin-bottom: 10px;
      }
```

```
        button {
            background-color: #4CAF50;
            color: white;
            padding: 10px 15px;
            border: none;
            border-radius: 4px;
            cursor: pointer;
            margin-right: 10px;
        }
        button:hover {
            background-color: #45a049;
        }
        #result {
            margin-top: 20px;
            padding: 10px;
            background-color: #e7e7e7;
            border-radius: 4px;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>JSON/CSV Converter and Hash Generator</h1>
        <h2>a) Convert JSON to JavaScript Object</h2>
        <textarea id="jsonInput" placeholder="Enter JSON here"></textarea>
        <button onclick="convertJsonToObject()">Convert to Object</button>
        <h2>b) Convert JSON to Date</h2>
        <textarea id="jsonDateInput" placeholder='Enter JSON date string (e.g.,
            {"date": "2023-05-15T12:00:00Z"})'></textarea>
        <button onclick="convertJsonToDate()">Convert to Date</button>
        <h2>c) Convert JSON to CSV and CSV to JSON</h2>
        <textarea id="dataInput" placeholder="Enter JSON or CSV
            here"></textarea>
        <button onclick="convertJsonToCsv()">JSON to CSV</button>
        <button onclick="convertCsvToJson()">CSV to JSON</button>
        <h2>d) Create Hash from String</h2>
        <textarea id="hashInput" placeholder="Enter string to
            hash"></textarea>
        <button onclick="createHash()">Generate Hash</button>
        <div id="result"></div>
    </div>
```

```
<script>
function convertJsonToObject() {
    try {
        const jsonInput = document.getElementById('jsonInput').value;
        const jsObject = JSON.parse(jsonInput);
        document.getElementById('result').innerText = 'Converted Object: ' +
        JSON.stringify(jsObject, null, 2);
    }
    catch (error) {
        document.getElementById('result').innerText = 'Error: ' +
        error.message;
    }
}
function convertJsonToDate() {
    try {
        const jsonInput = document.getElementById('jsonDateInput').value;
        const jsObject = JSON.parse(jsonInput);
        const date = new Date(jsObject.date);
        document.getElementById('result').innerText = 'Converted Date: ' +
        date.toString();
    }
    catch (error) {
        document.getElementById('result').innerText = 'Error: ' +
        error.message;
    }
}
function convertJsonToCsv() {
    try {
        const jsonInput = document.getElementById('dataInput').value;
        const jsObject = JSON.parse(jsonInput);
        const headers = Object.keys(jsObject[0]);
        const csvRows = [
        headers.join(','),
        ...jsObject.map(row =>headers.map(fieldName
        =>JSON.stringify(row[fieldName])).join(','))
        ];
    const csvString = csvRows.join('\n');
        document.getElementById('result').innerText = 'Converted CSV:\n' +
        csvString;
    }
```

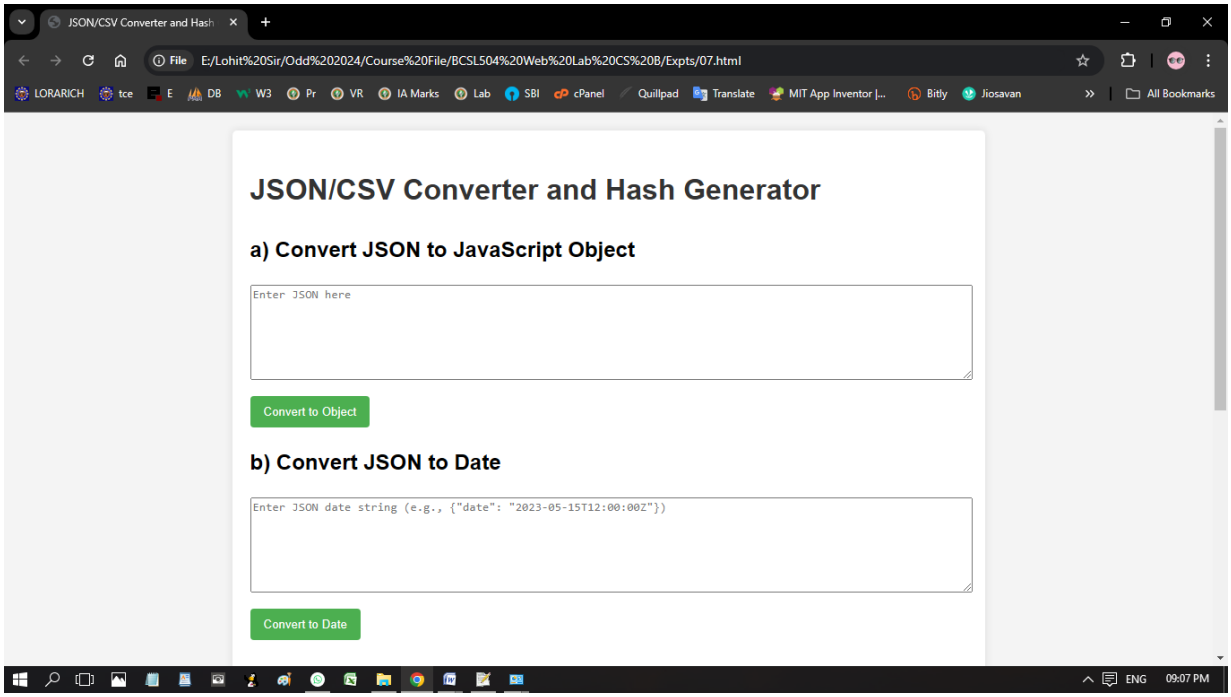
```
            catch (error) {
                document.getElementById('result').innerText = 'Error: ' +
                error.message;
            }
        }
        function convertCsvToJson() {
            try {
                const csvInput = document.getElementById('dataInput').value;
                const lines = csvInput.split('\n');
                const headers = lines[0].split(',');
                const jsonArray = lines.slice(1).map(line => {
                const values = line.split(',');
                return headers.reduce((obj, header, index) => {
                obj[header] = values[index];
                return obj;
                }, {});
                });
                document.getElementById('result').innerText = 'Converted JSON:\n'
                + JSON.stringify(jsonArray, null, 2);
            }
            catch (error) {
                document.getElementById('result').innerText = 'Error: ' +
                error.message;
            }
        }
        function createHash() {
            try {
                const input = document.getElementById('hashInput').value;
                const hash = CryptoJS.SHA256(input);
                document.getElementById('result').innerText = 'Generated Hash
                (SHA-256): ' + hash;
            }
            catch (error) {
                document.getElementById('result').innerText = 'Error: ' +
                error.message;
            }
        }
        </script>
    </body>
</html>
```

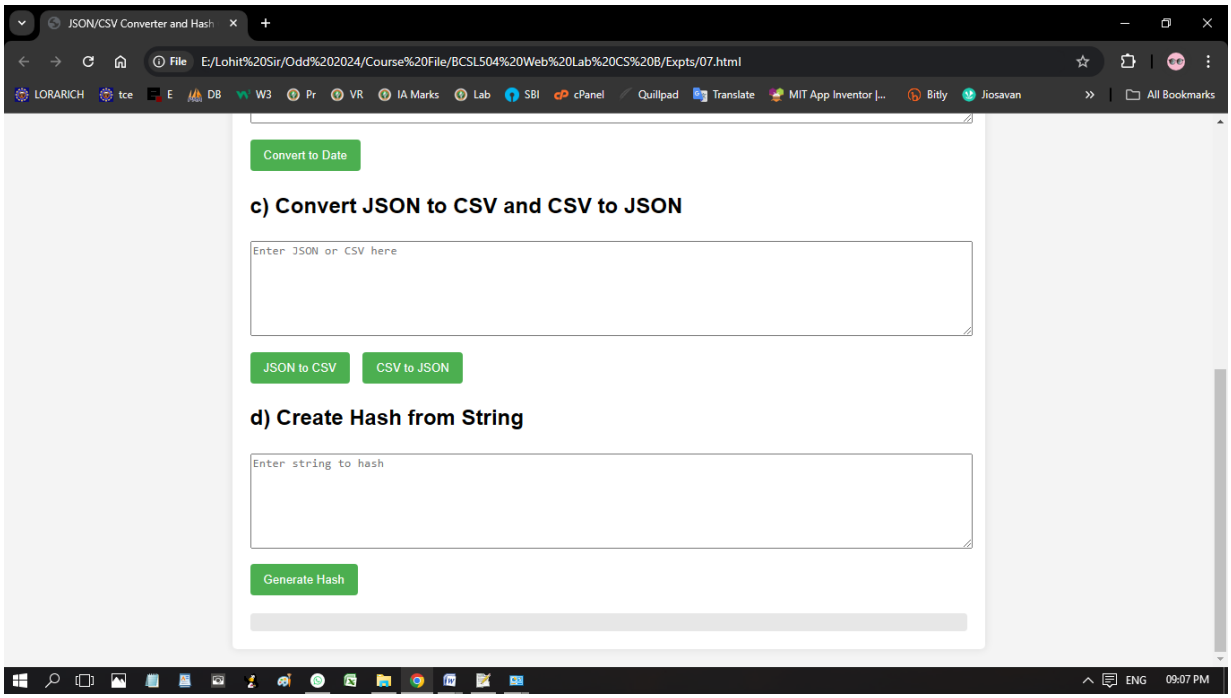
Explanation:
This is a tool that helps convert data between different formats (JSON and CSV) and create hash values from text. It's designed to be user-friendly and educational.
Main features:

a) Convert JSON to JavaScript Object
b) Convert JSON to Date
c) Convert JSON to CSV and CSV to JSON
d) Create a Hash from a String

Structure of the page:
The page is divided into sections, each with a text area for input and a button to perform the conversion or operation.
How it works:
Users can paste their data into the text areas and click the corresponding buttons to see the results. The converted data or hash appears in a result section at the bottom of the page.
Technologies used:
HTML: For structuring the webpage
CSS: For styling and making the page look nice
JavaScript: For performing the conversions and interactions
CryptoJS library: For creating hash values
Now, let's explain each main feature in simple terms:
a) Convert JSON to JavaScript Object:
This takes JSON (a way to represent data as text) and turns it into a JavaScript object that can be used in code.
b) Convert JSON to Date:
This takes a date written in JSON format and converts it into a regular date that's easier to read and use.
c) Convert JSON to CSV and CSV to JSON:
JSON to CSV: This takes data in JSON format and turns it into CSV (Comma-Separated Values), which is like a simple spreadsheet.
CSV to JSON: This does the opposite, taking CSV data and turning it into JSON.
d) Create Hash from String:
This takes any text you enter and creates a unique "fingerprint" (hash) for that text using a method called SHA-256.

OUTPUT:


JSON/CSV Converter and Hash
File E:/Lohit%20Sir/Odd%202024/Course%20File/BCSL504%20Web%20Lab%20CS%20B/Expts/07.html
JSON/CSV Converter and Hash Generator
a) Convert JSON to JavaScript Object
Enter JSON here
Convert to Object
b) Convert JSON to Date
Enter JSON date string (e.g., {"date": "2023-05-15T12:00:00Z"})
Convert to Date
ENG
09:07 PM


JSON/CSV Converter and Hash
File E:/Lohit%20Sir/Odd%202024/Course%20File/BCSL504%20Web%20Lab%20CS%20B/Expts/07.html
Convert to Date
c) Convert JSON to CSV and CSV to JSON
Enter JSON or CSV here
JSON to CSV
CSV to JSON
d) Create Hash from String
Enter string to hash
Generate Hash
ENG
09:07 PM

Experiment – 8(a)

Develop a PHP program (with HTML/CSS) to keep track of the number of visitors visiting the webpage and to display this count of visitors, with relevant headings.

SOLUTION:

Here's a PHP program that tracks the number of visitors visiting a web page, along with HTML and CSS for a simple design. This program uses a file-based approach to store the count of visitors, which increments each time the page is loaded.

1. Install a Local Server (If Not Done Already):

PHP programs require a server to run. You can use local server software such as XAMPP, WAMP, MAMP or EasyPHP.

These are free, cross-platform server package that includes Apache (for hosting), MySQL (for databases), and PHP.

Download and install any one from their website.

2. Set Up Your Project Directory:

Once installed, create a folder in the web directory. Name your folder something like Program_8a. Place all your project files in this folder.

3. Write the PHP Program:

Create a PHP file called index.php inside the folder. Add the following PHP, HTML, and CSS code.

4. Create the Visitor Count File:

The program will create the file visitor_count.txt automatically if it doesn't exist. You don't need to create it manually but ensure that the web server has permission to write to the directory.

5. Access the PHP Program:

Open a browser and navigate to the following URL:

http://localhost/visitor_tracker/index.php.

Every time you visit the page, the visitor count will increase and be displayed on the screen.

```
<?php
// Path to the file where the visitor count is stored
$counter_file = "visitor_count.txt";
// Check if the file exists, and if not, create it and set the count to 0
if (! file_exists($counter_file)) {
file_put_contents ($counter_file, 0);
}
// Read the current count from the file
```

```
$current_count = (int)file_get_contents($counter_file);
// Increment the visitor count
$current_count++;
// Save the updated count back to the file
file_put_contents ($counter_file, $current_count);
?>
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Visitor Counter</title>
<style>
body {
font-family: Arial, sans-serif;
background-color: #f0f8ff;
display: flex;
flex-direction: column;
justify-content: center;
align-items: center;
height: 100vh;
margin: 0;
}
.container {
text-align: center;
background-color: #ffffff;
padding: 30px;
border-radius: 8px;
box-shadow: 0px 4px 8px rgba(0, 0, 0, 0.1);
}
h1 {
color: #333;
font-size: 2.5rem;
margin-bottom: 10px;
}
p {
font-size: 1.5rem;
color: #555;
}
.counter {
font-size: 3rem;
font-weight: bold;
```

```
color: #2e8b57;
}
</style>
</head>
<body>
<div class="container">
<h1>Welcome to Our Website!</h1>
<p>You are visitor number:</p>
<div class="counter"><?php echo $current_count; ?></div>
</div>
</body>
</html>
```

Explanation of the Program:

PHP Section:

$counter_file = "visitor_count.txt";

This specifies the file that stores the visitor count.

file_exists()

This checks if the file visitor_count.txt exists. If it doesn't, the file is created with an initial value of 0 using file_put_contents().

file_get_contents()

Reads the current visitor count from the file.

$current_count++

Increments the visitor count every time the page is visited.

file_put_contents()

Writes the updated count back to the file.

HTML and CSS Section:

The HTML defines the structure of the page, including a heading (<h1>Welcome to Our Website!</h1>), a message (You are visitor number:), and the visitor count ( ).

The CSS is used to style the page, making it visually appealing with centered content, colors, and fonts. It ensures the page looks neat and professional.

OUTPUT:

Experiment – 9

Develop jQuery script (with HTML/CSS) for:

a. **Appends the content at the end of the existing paragraph and list.**
b. **Change the state of the element with CSS style using animate() method**
c. **Change the color of any div that is animated.**

Develop jQuery script with HTML/CSS Program:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>jQuery Append, Animate, and Color Change Demo</title>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
    <style>
      body {
        font-family: Arial, sans-serif;
        line-height: 1.6;
        margin: 0;
        padding: 20px;
        background-color: #f4f4f4;
      }
      .container {
        max-width: 800px;
        margin: auto;
        background: white;
        padding: 20px;
        border-radius: 5px;
        box-shadow: 0 0 10px rgba(0,0,0,0.1);
      }
      h1, h2 {
        color: #333;
      }
      .box {
        width: 100px;
        height: 100px;
        background-color: #3498db;
        margin: 20px 0;
      }
```

```
        button {
            padding: 10px 15px;
            background-color: #2ecc71;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
            margin-right: 10px;
        }
        button:hover {
            background-color: #27ae60;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>jQuery Demonstration</h1>
        <h2>a. Append Content</h2>
        <p id="existingParagraph">This is an existing paragraph. </p>
        <ul id="existingList">
            <li>Existing item 1</li>
            <li>Existing item 2</li>
        </ul>
        <button id="appendButton">Append Content</button>
        <h2>b. Animate Element</h2>
        <div id="animateBox" class="box"></div>
        <button id="animateButton">Animate Box</button>
        <h2>c. Change Color of Animated Div</h2>
        <div id="colorBox" class="box"></div>
        <button id="colorAnimateButton">Animate and Change Color</button>
    </div>
    <script>
        $(document).ready(function() {
            // a. Append content
            $("#appendButton").click(function() {
                $("#existingParagraph").append("This content is appended.");
                $("#existingList").append("<li>Appended item</li>");
            });
```

```
            // b. Animate element
            $("#animateButton").click(function() {
                $("#animateBox").animate({
                width: "200px",
                height: "200px",
                opacity: 0.5
                }, 1000);
            });
            // c. Animate and change color
            $("#colorAnimateButton").click(function() {
                $("#colorBox").animate({
                    width: "200px",
                    height: "200px"
                }, {
                    duration: 1000,
                    step: function(now, fx) {
                        if (fx.prop === "width") {
                            $(this).css("background-color",
                            `rgb(${Math.round(now)}, 52, 219)`);
                        }
                    }
                });
            });
        });
    </script>
  </body>
</html>
```

Explanation:

Purpose of the webpage:

This is a demonstration page showing how to use jQuery, a popular JavaScript library, to manipulate web page elements dynamically. It includes examples of appending content, animating elements, and changing colors.

Structure of the page:

The page is divided into three main sections, each demonstrating a different jQuery functionality:

a) Append Content
b) Animate Element
c) Change Color of Animated Div

Technologies used:

HTML: For structuring the webpage

CSS: For styling the page elements

jQuery: A JavaScript library for easier DOM manipulation and animations

Key components and their functions:

a) Append Content:

This section has an existing paragraph and a list.

When you click the "Append Content" button, it adds text to the paragraph and a new item to the list.

How it works: jQuery selects the elements by their IDs and uses the .append() method to add new content.

b) Animate Element:

This section has a blue square (box).

Clicking the "Animate Box" button makes the box grow larger and more transparent.

How it works: jQuery uses the .animate() method to gradually change the box's width, height, and opacity over 1 second (1000 milliseconds).

c) Change Color of Animated Div:

This section also has a blue square.

Clicking the "Animate and Change Color" button makes the box grow larger while changing its color from blue to red.

How it works: jQuery animates the box's size and simultaneously updates its background color based on the current width during the animation.

jQuery setup:

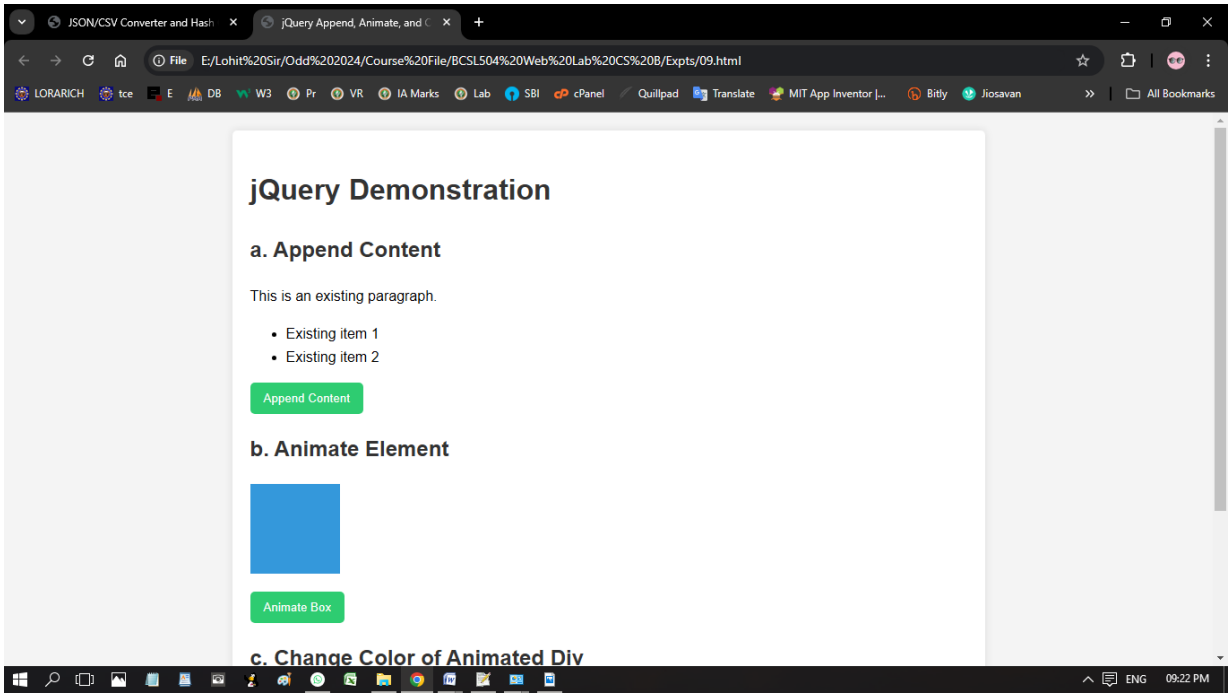
The script uses $(document).ready(function() { ... }) to ensure that the jQuery code only runs after the page has fully loaded.

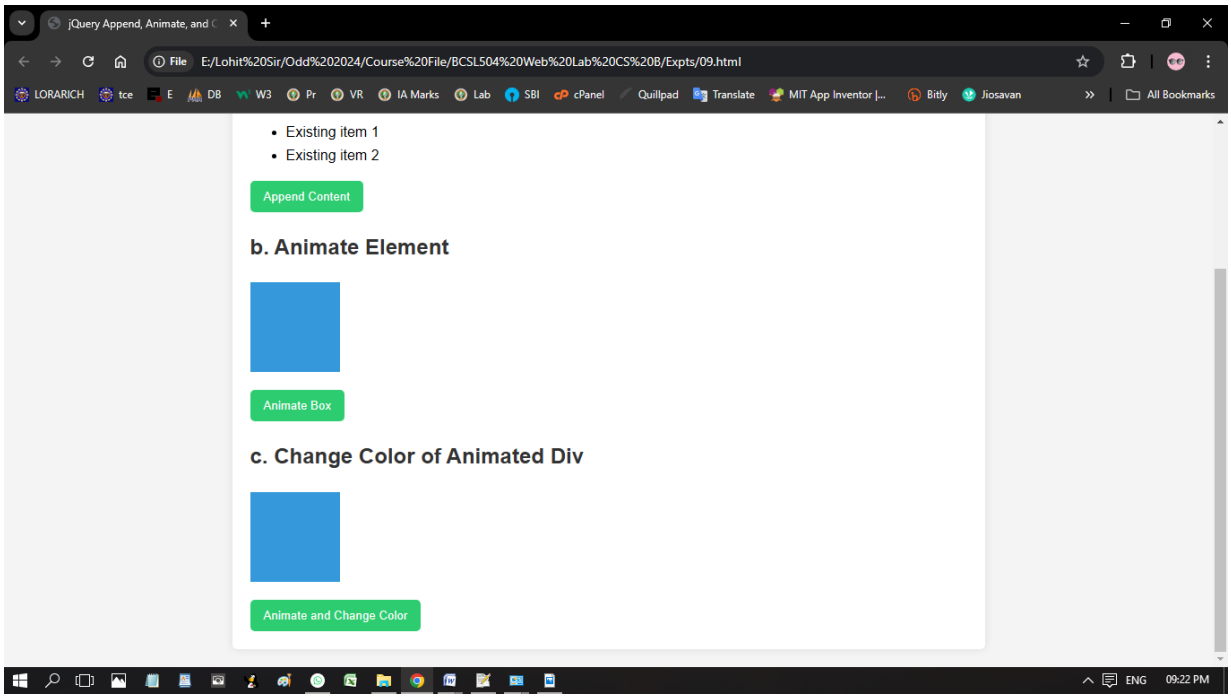
Event handling:

Each button has a click event handler set up using jQuery's .click() method.

When a button is clicked, it triggers the corresponding function to perform the animation or content append.

OUTPUT:


JSON/CSV Converter and Hash
jQuery Append, Animate, and
E:/Lohit%20Sir/Odd%202024/Course%20File/BCSL504%20Web%20Lab%20CS%20B/Expts/09.html
jQuery Demonstration
a. Append Content
This is an existing paragraph.
Existing item 1
Existing item 2
Append Content
b. Animate Element
Animate Box
c. Change Color of Animated Div


jQuery Append, Animate, and
E:/Lohit%20Sir/Odd%202024/Course%20File/BCSL504%20Web%20Lab%20CS%20B/Expts/09.html
Existing item 1
Existing item 2
Append Content
b. Animate Element
Animate Box
c. Change Color of Animated Div
Animate and Change Color

Experiment – 10

Develop a JavaScript program with Ajax (with HTML/CSS) for:

a. **Use ajax() method (without Jquery) to add the text content from the text file by sending ajax request.**
b. **Use ajax() method (with Jquery) to add the text content from the text file by sending ajax request.**
c. **Illustrate the use of getJSON() method in jQuery**
d. **Illustrate the use of parseJSON() method to display JSON values.**

JavaScript program with Ajax (with HTML/CSS):

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Ajax Demo Program</title>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <script
      src="https://cdnjs.cloudflare.com/ajax/libs/jquery/3.6.0/jquery.min.js"></scr
      ipt>
    <style>
      body {
          font-family: Arial, sans-serif;
          line-height: 1.6;
          margin: 0;
          padding: 20px;
          background-color: #f4f4f4;
      }
      .container {
          max-width: 800px;
          margin: auto;
          background: white;
          padding: 20px;
          border-radius: 5px;
          box-shadow: 0 0 10px rgba(0,0,0,0.1);
      }
      h1 {
          color: #333;
      }
      h2 {
          color: #666;
      }
```

```
        button {
            background-color: #4CAF50;
            border: none;
            color: white;
            padding: 10px 20px;
            text-align: center;
            text-decoration: none;
            display: inline-block;
            font-size: 16px;
            margin: 4px 2px;
            cursor: pointer;
            border-radius: 4px;
        }
        pre {
            background-color: #f8f8f8;
            border: 1px solid #ddd;
            border-radius: 4px;
            padding: 10px;
            white-space: pre-wrap;
            word-wrap: break-word;
        }
    </style>
</head>
<body>
    <div class="container">
    <h1>Ajax Demo Program</h1>
        <h2>a. Ajax-like operation without jQuery</h2>
        <button onclick="operationWithoutJQuery()">Perform Operation
         (without jQuery)</button>
        <pre id="result-a"></pre>
        <h2>b. Ajax-like operation with jQuery</h2>
        <button onclick="operationWithJQuery()">Perform Operation (with
        jQuery)</button>
        <pre id="result-b"></pre>
        <h2>c. jQuery-like getJSON() method</h2>
        <button onclick="getJSONOperation()">Get JSON</button>
        <pre id="result-c"></pre>
        <h2>d. jQuery parseJSON() method</h2>
        <button onclick="parseJSONExample()">Parse JSON</button>
        <pre id="result-d"></pre>
    </div>
```

```
<script>
    // Simulated data
    const simulatedData = {
        text: "This is a sample text from a simulated server response.",
        json: {
            name: "John Doe",
            age: 30,
            city: "New York"
        }
    };
    a. Ajax-like operation without jQuery
    function operationWithoutJQuery() {
        setTimeout(function() {
            document.getElementById("result-a").textContent =
            simulatedData.text;
        }, 500);
    }
    // b. Ajax-like operation with jQuery
    function operationWithJQuery() {
        $.Deferred(function(deferred) {
            setTimeout(function() {
                deferred.resolve(simulatedData.text);
            }, 500);
        }).done(function(result) {
            $("#result-b").text(result);
        });
    }
    // c. jQuery-like getJSON() method
    function getJSONOperation() {
        $.Deferred(function(deferred) {
            setTimeout(function() {
                deferred.resolve(simulatedData.json);
            }, 500);
        }).done(function(result) {
            $("#result-c").text(JSON.stringify(result, null, 2));
        });
    }
```

```
            // d. jQuery parseJSON() method
            function parseJSONExample() {
                var jsonString = JSON.stringify(simulatedData.json);
                var jsonObject = $.parseJSON(jsonString);
                $("#result-d").text(JSON.stringify(jsonObject, null, 2));
            }
        </script>
    </body>
</html>
```

Explanation:

Purpose of the webpage:

This is a demonstration page showing different methods of handling asynchronous operations and JSON data, both with and without jQuery. It's designed to help students understand the differences between these approaches.

The page is divided into four main sections, each demonstrating a different technique:

a. Ajax-like operation without jQuery
b. Ajax-like operation with jQuery
c. jQuery-like getJSON() method
d. jQuery parseJSON() method

Technologies used:

HTML: For structuring the webpage
CSS: For styling the page elements
JavaScript: For performing operations without jQuery
jQuery: A JavaScript library for easier Ajax and JSON handling

a. Ajax-like operation without jQuery:

- This simulates an asynchronous operation using plain JavaScript.
- It uses setTimeout() to mimic a delay in receiving data from a server.
- The result is displayed using vanilla JavaScript DOM manipulation.

b. Ajax-like operation with jQuery:

- This demonstrates how to use jQuery's Deferred object to handle asynchronous operations.
- It also uses setTimeout() to simulate a delay.
- The result is displayed using jQuery's DOM manipulation methods.

c. jQuery-like getJSON() method:

- This simulates fetching JSON data from a server using jQuery's Deferred object.
- It demonstrates how to handle and display JSON data.

d. jQuery parseJSON() method:

- This shows how to parse a JSON string into a JavaScript object using jQuery.
- It then displays the parsed object.
- The code uses a simulatedData object to mimic data that would typically come from a server.
- This allows the demonstration to work without an actual server connection.

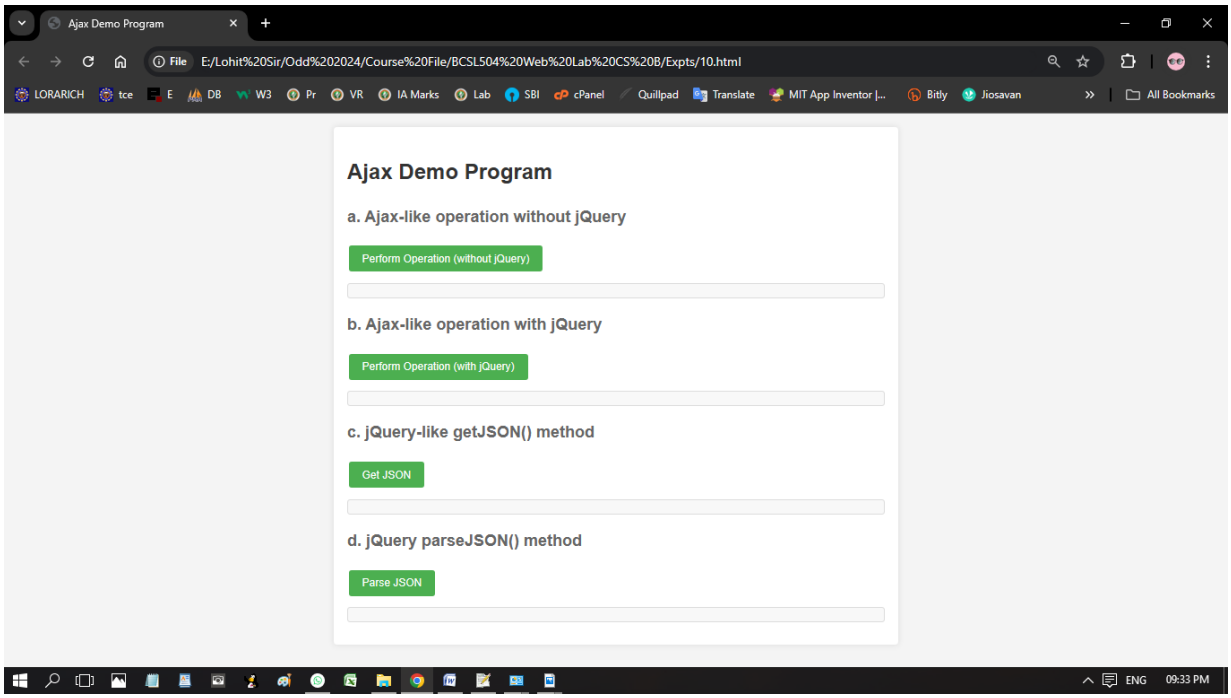
Asynchronous Operations:

- All the operations (except the parseJSON example) use setTimeout() to simulate the delay that would occur when fetching data from a real server.
- This helps students understand the concept of asynchronous programming.

jQuery Usage:

- The code demonstrates how jQuery can simplify Ajax-like operations and JSON handling.
- It shows the difference between using vanilla JavaScript and jQuery for similar tasks.

OUTPUT:



Appendix – I
HTML Tag List by Category

Basic HTML

Tag	Description
<!DOCTYPE>	Defines the document type
<html>	Defines an HTML document
<head>	Contains metadata/information for the document
<title>	Defines a title for the document
<body>	Defines the document's body
<h1> to <h6>	Defines HTML headings
<p>	Defines a paragraph
 	Inserts a single line break
<hr>	Defines a thematic change in the content
<!--...-->	Defines a comment

Formatting

Tag	Description
<acronym>	Not supported in HTML5. Use <abbr> instead. Defines an acronym
<abbr>	Defines an abbreviation or an acronym
<address>	Defines contact information for the author/owner of a document/article
<b>	Defines bold text
<bdi>	Isolates a part of text that might be formatted in a different direction from other text outside it
<bdo>	Overrides the current text direction
<big>	Not supported in HTML5. Use CSS instead. Defines big text
<blockquote>	Defines a section that is quoted from another source
	Not supported in HTML5. Use CSS instead. Defines centered text

<cite>	Defines the title of a work
<code>	Defines a piece of computer code
<del>	Defines text that has been deleted from a document
<dfn>	Specifies a term that is going to be defined within the content
	Defines emphasized text
<font>	Not supported in HTML5. Use CSS instead. Defines font, color, and size for text
<i>	Defines a part of text in an alternate voice or mood
<ins>	Defines a text that has been inserted into a document
<kbd>	Defines keyboard input
<mark>	Defines marked/highlighted text
<meter>	Defines a scalar measurement within a known range (a gauge)
<pre>	Defines preformatted text
<progress>	Represents the progress of a task
<q>	Defines a short quotation
<rp>	Defines what to show in browsers that do not support ruby annotations
<rt>	Defines an explanation/pronunciation of characters (for East Asian typography)
<ruby>	Defines a ruby annotation (for East Asian typography)
<s>	Defines text that is no longer correct
<samp>	Defines sample output from a computer program
<small>	Defines smaller text
<strike>	Not supported in HTML5. Use <del> or <s> instead. Defines strikethrough text
	Defines important text
<sub>	Defines subscripted text
<sup>	Defines superscripted text

<template>	Defines a container for content that should be hidden when the page loads
<time>	Defines a specific time (or datetime)
<tt>	Not supported in HTML5. Use CSS instead. Defines teletype text
<u>	Defines some text that is unarticulated and styled differently from normal text
<var>	Defines a variable
<wbr>	Defines a possible line-break

Forms and Input

Tag	Description
<form>	Defines an HTML form for user input
<input>	Defines an input control
<textarea>	Defines a multiline input control (text area)
<button>	Defines a clickable button
<select>	Defines a drop-down list
<optgroup>	Defines a group of related options in a drop-down list
<option>	Defines an option in a drop-down list
<label>	Defines a label for an <input> element
<fieldset>	Groups related elements in a form
<legend>	Defines a caption for a <fieldset> element
<datalist>	Specifies a list of pre-defined options for input controls
<output>	Defines the result of a calculation

Frames

Tag	Description
<frame>	Not supported in HTML5. Defines a window (a frame) in a frameset
<frameset>	Not supported in HTML5. Defines a set of frames
<noframes>	Not supported in HTML5. Defines an alternate content for users that do not support frames
<iframe>	Defines an inline frame

Images

Tag	Description
<img>	Defines an image
<map>	Defines a client-side image map
<area>	Defines an area inside an image map
<canvas>	Used to draw graphics, on the fly, via scripting (usually JavaScript)
<figcaption>	Defines a caption for a <figure> element
<figure>	Specifies self-contained content
<picture>	Defines a container for multiple image resources
<svg>	Defines a container for SVG graphics

Audio / Video

Tag	Description
<audio>	Defines sound content
<source>	Defines multiple media resources for media elements (<video>, <audio> and <picture>)
<track>	Defines text tracks for media elements (<video> and <audio>)
<video>	Defines a video or movie

Links

Tag	Description
<a>	Defines a hyperlink
<link>	Defines the relationship between a document and an external resource (most used to link to style sheets)
<nav>	Defines navigation links

Lists

Tag	Description
	Defines an unordered list
	Defines an ordered list
	Defines a list item
<dir>	Not supported in HTML5. Use instead. Defines a directory list
<dl>	Defines a description list
<dt>	Defines a term/name in a description list
<dd>	Defines a description of a term/name in a description list

Tables

Tag	Description
<table>	Defines a table
<caption>	Defines a table caption
<th>	Defines a header cell in a table
<tr>	Defines a row in a table
<td>	Defines a cell in a table
<thead>	Groups the header content in a table
<tbody>	Groups the body content in a table
<tfoot>	Groups the footer content in a table
<col>	Specifies column properties for each column within a <colgroup> element

<colgroup>	Specifies a group of one or more columns in a table for formatting

Styles and Semantics

Tag	Description
<style>	Defines style information for a document
	Defines a section in a document
	Defines a section in a document
<header>	Defines a header for a document or section
<footer>	Defines a footer for a document or section
<main>	Specifies the main content of a document
<section>	Defines a section in a document
<article>	Defines an article
<aside>	Defines content aside from the page content
<details>	Defines additional details that the user can view or hide
<dialog>	Defines a dialog box or window
<summary>	Defines a visible heading for a <details> element
<data>	Adds a machine-readable translation of a given content

Meta Info

Tag	Description
<head>	Defines information about the document
<meta>	Defines metadata about an HTML document
<base>	Specifies the base URL/target for all relative URLs in a document
<basefont>	Not supported in HTML5. Use CSS instead. Specifies a default color, size, and font for all text in a document

Programming

Tag	Description
<script>	Defines a client-side script
<noscript>	Defines an alternate content for users that do not support client-side scripts
<applet>	Not supported in HTML5. Use <embed> or <object> instead. Defines an embedded applet
<embed>	Defines a container for an external (non-HTML) application
<object>	Defines an embedded object
<param>	Defines a parameter for an object

Appendix – II
HTML Attributes

The table below lists all HTML attributes and what elements they can be used within:

Attribute	Belongs to	Description
accept	<input>	Specifies the types of files that the server accepts (only for type="file")
accept-charset	<form>	Specifies the character encodings that are to be used for the form submission
accesskey	Global Attributes	Specifies a shortcut key to activate/focus an element
action	<form>	Specifies where to send the form-data when a form is submitted
align	Unsupport HTML5	Specifies the alignment according to surrounding elements. Use CSS instead
alt	<area> <img> <input>	Specifies an alternate text when the original element fails to display
async	<script>	Specifies that the script is executed asynchronously (only for external scripts)
autocomplete	<form> <input>	Specifies whether the <form> or the <input> element should have autocomplete enabled
autofocus	<button> <input> <select> <textarea>	Specifies that the element should automatically get focus when the page loads
autoplay	<audio> <video>	Specifies that the audio/video will start playing as soon as it is ready
bgcolor	Unsupport HTML5	Specifies the background color of an element. Use CSS instead
border	Unsupport HTML5	Specifies the width of the border of an element. Use CSS instead
charset	<meta> <script>	Specifies the character encoding

Attribute	Belongs to	Description
checked	<input>	Specifies that an <input> element should be pre-selected when the page loads (for type="checkbox" or type="radio")
cite	<blockquote> <del> <ins> <q>	Specifies a URL which explains the quote/deleted/inserted text
class	Global Attributes	Specifies one or more classnames for an element (refers to a class in a style sheet)
color	Unsupport HTML5	Specifies the text color of an element. Use CSS instead
cols	<textarea>	Specifies the visible width of a text area
colspan	<td> <th>	Specifies the number of columns a table cell should span
content	<meta>	Gives the value associated with the http-equiv or name attribute
contenteditable	Global Attributes	Specifies whether the content of an element is editable or not
controls	<audio> <video>	Specifies that audio/video controls should be displayed (such as a play/pause button etc)
coords	<area>	Specifies the coordinates of the area
data	<object>	Specifies the URL of the resource to be used by the object
data-*	Global Attributes	Used to store custom data private to the page or application
datetime	<del> <ins> <time>	Specifies the date and time
default	<track>	Specifies that the track is to be enabled if the user's preferences do not indicate that another track would be more appropriate

Attribute	Belongs to	Description
defer	<script>	Specifies that the script is executed when the page has finished parsing (only for external scripts)
dir	Global Attributes	Specifies the text direction for the content in an element
dirname	<input> <textarea>	Specifies that the text direction will be submitted
disabled	<button> <fieldset> <input> <optgroup> <option> <select> <textarea>	Specifies that the specified element/group of elements should be disabled
download	<a> <area>	Specifies that the target will be downloaded when a user clicks on the hyperlink
draggable	Global Attributes	Specifies whether an element is draggable or not
enctype	<form>	Specifies how the form-data should be encoded when submitting it to the server (only for method="post")
for	<label> <output>	Specifies which form element(s) a label/calculation is bound to
form	<button> <fieldset> <input> <label> <meter> <object> <output> <select> <textarea>	Specifies the name of the form the element belongs to
formaction	<button> <input>	Specifies where to send the form-data when a form is submitted. Only for type="submit"
headers	<td> <th>	Specifies one or more headers cells a cell is related to

Attribute	Belongs to	Description

height	<canvas> <embed> <iframe> <img> <input> <object>, <video>	Specifies the height of the element
hidden	Global Attributes	Specifies that an element is not yet or is no longer relevant
high	<meter>	Specifies the range that is considered to be a high value
href	<a> <area> <base> <link>	Specifies the URL of the page the link goes to
hreflang	<a> <area> <link>	Specifies the language of the linked document
http-equiv	<meta>	Provides an HTTP header for the information/value of the content attribute
id	Global Attributes	Specifies a unique id for an element
ismap	<img>	Specifies an image as a server-side image map
kind	<track>	Specifies the kind of text track
label	<track> <option> <optgroup>	Specifies the title of the text track
lang	Global Attributes	Specifies the language of the element's content
list	<input>	Refers to a <datalist> element that contains pre-defined options for an <input> element
loop	<audio> <video>	Specifies that the audio/video will start over again every time it is finished

Attribute	Belongs to	Description
low	<meter>	Specifies the range that is considered to be a low value
max	<input>	Specifies the maximum value

	<meter> <progress>	
maxlength	<input> <textarea>	Specifies the maximum number of characters allowed in an element
media	<a> <area> <link> <source> <style>	Specifies what media/device the linked document is optimized for
method	<form>	Specifies the HTTP method to use when sending form-data
min	<input> <meter>	Specifies a minimum value
multiple	<input> <select>	Specifies that a user can enter more than one value
muted	<video> <audio>	Specifies that the audio output of the video should be muted
name	<button> <fieldset> <form> <iframe> <input> <map> <meta> <object> <output> <param> <select> <textarea>	Specifies the name of the element
novalidate	<form>	Specifies that the form should not be validated when submitted

Attribute	Belongs to	Description
onabort	<audio> <embed> <img>	Script to be run on abort

	<object> <video>	
onafterprint	<body>	Script to be run after the document is printed
onbeforeprint	<body>	Script to be run before the document is printed
onbeforeunload	<body>	Script to be run when the document is about to be unloaded
onblur	All visible elements.	Script to be run when the element loses focus
oncanplay	<audio> <embed> <object> <video>	Script to be run when a file is ready to start playing (when it has buffered enough to begin)
oncanplaythrough	<audio> <video>	Script to be run when a file can be played all the way to the end without pausing for buffering
onchange	All visible elements.	Script to be run when the value of the element is changed
onclick	All visible elements.	Script to be run when the element is being clicked
oncontextmenu	All visible elements.	Script to be run when a context menu is triggered
oncopy	All visible elements.	Script to be run when the content of the element is being copied
oncuechange	<track>	Script to be run when the cue changes in a <track> element
oncut	All visible elements.	Script to be run when the content of the element is being cut
ondblclick	All visible elements.	Script to be run when the element is being double-clicked

Attribute	Belongs to	Description
ondrag	All visible elements.	Script to be run when the element is being dragged

ondragend	All visible elements.	Script to be run at the end of a drag operation
ondragenter	All visible elements.	Script to be run when an element has been dragged to a valid drop target
ondragleave	All visible elements.	Script to be run when an element leaves a valid drop target
ondragover	All visible elements.	Script to be run when an element is being dragged over a valid drop target
ondragstart	All visible elements.	Script to be run at the start of a drag operation
ondrop	All visible elements.	Script to be run when dragged element is being dropped
ondurationchange	<audio> <video>	Script to be run when the length of the media changes
onemptied	<audio> <video>	Script to be run when something bad happens and the file is suddenly unavailable (like unexpectedly disconnects)
onended	<audio> <video>	Script to be run when the media has reach the end (a useful event for messages like "thanks for listening")
onerror	<audio> <body> <embed> <img> <object> <script> <style> <video>	Script to be run when an error occurs
onfocus	All visible elements.	Script to be run when the element gets focus
onhashchange	<body>	Script to be run when there has been changes to the anchor part of the a URL

Attribute	Belongs to	Description
oninput	All visible elements.	Script to be run when the element gets user input

oninvalid	All visible elements.	Script to be run when the element is invalid
onkeydown	All visible elements.	Script to be run when a user is pressing a key
onkeypress	All visible elements.	Script to be run when a user presses a key
onkeyup	All visible elements.	Script to be run when a user releases a key
onload	<body> <iframe> <img> <input> <link> <script> <style>	Script to be run when the element is finished loading
onloadeddata	<audio> <video>	Script to be run when media data is loaded
onloadedmetadata	<audio> <video>	Script to be run when meta data (like dimensions and duration) are loaded
onloadstart	<audio> <video>	Script to be run just as the file begins to load before anything is actually loaded
onmousedown	All visible elements.	Script to be run when a mouse button is pressed down on an element
onmousemove	All visible elements.	Script to be run as long as the mouse pointer is moving over an element
onmouseout	All visible elements.	Script to be run when a mouse pointer moves out of an element
onmouseover	All visible elements.	Script to be run when a mouse pointer moves over an element
onmouseup	All visible elements.	Script to be run when a mouse button is released over an element

Attribute	Belongs to	Description
onmousewheel	All visible elements.	Script to be run when a mouse wheel is being scrolled over an element

onoffline	<body>	Script to be run when the browser starts to work offline
ononline	<body>	Script to be run when the browser starts to work online
onpagehide	<body>	Script to be run when a user navigates away from a page
onpageshow	<body>	Script to be run when a user navigates to a page
onpaste	All visible elements.	Script to be run when the user pastes some content in an element
onpause	<audio> <video>	Script to be run when the media is paused either by the user or programmatically
onplay	<audio> <video>	Script to be run when the media has started playing
onplaying	<audio> <video>	Script to be run when the media has started playing
onpopstate	<body>	Script to be run when the window's history changes.
onprogress	<audio> <video>	Script to be run when the browser is in the process of getting the media data
onratechange	<audio> <video>	Script to be run each time the playback rate changes (like when a user switches to a slow motion or fast forward mode).
onreset	<form>	Script to be run when a reset button in a form is clicked.
onresize	<body>	Script to be run when the browser window is being resized.
onscroll	All visible elements.	Script to be run when an element's scrollbar is being scrolled
onsearch	<input>	Script to be run when the user writes something in a search field (for <input type="search">)

Attribute	Belongs to	Description
onseeked	<audio> <video>	Script to be run when the seeking attribute is set to false indicating that seeking has ended

onseeking	<audio> <video>	Script to be run when the seeking attribute is set to true indicating that seeking is active
onselect	All visible elements.	Script to be run when the element gets selected
onstalled	<audio> <video>	Script to be run when the browser is unable to fetch the media data for whatever reason
onstorage	<body>	Script to be run when a Web Storage area is updated
onsubmit	<form>	Script to be run when a form is submitted
onsuspend	<audio> <video>	Script to be run when fetching the media data is stopped before it is completely loaded for whatever reason
ontimeupdate	<audio> <video>	Script to be run when the playing position has changed (like when the user fast forwards to a different point in the media)
ontoggle	<details>	Script to be run when the user opens or closes the <details> element
onunload	<body>	Script to be run when a page has unloaded (or the browser window has been closed)
onvolumechange	<audio> <video>	Script to be run each time the volume of a video/audio has been changed
onwaiting	<audio> <video>	Script to be run when the media has paused but is expected to resume (like when the media pauses to buffer more data)
onwheel	All visible elements.	Script to be run when the mouse wheel rolls up or down over an element
open	<details>	Specifies that the details should be visible (open) to the user
optimum	<meter>	Specifies what value is the optimal value for the gauge
pattern	<input>	Specifies a regular expression that an <input> element's value is checked against

Attribute	Belongs to	Description
placeholder	<input> <textarea>	Specifies a short hint that describes the expected value of the element

poster	<video>	Specifies an image to be shown while the video is downloading or until the user hits the play button
preload	<audio> <video>	Specifies if and how the author thinks the audio/video should be loaded when the page loads
readonly	<input> <textarea>	Specifies that the element is read-only
rel	<a> <area> <form> <link>	Specifies the relationship between the current document and the linked document
required	<input> <select> <textarea>	Specifies that the element must be filled out before submitting the form
reversed		Specifies that the list order should be descending (9 8 7...)
rows	<textarea>	Specifies the visible number of lines in a text area
rowspan	<td> <th>	Specifies the number of rows a table cell should span
sandbox	<iframe>	Enables an extra set of restrictions for the content in an <iframe>
scope	<th>	Specifies whether a header cell is a header for a column row or group of columns or rows
selected	<option>	Specifies that an option should be pre-selected when the page loads
shape	<area>	Specifies the shape of the area

Attribute	Belongs to	Description
size	<input> <select>	Specifies the width in characters (for <input>) or specifies the number of visible options (for <select>)

sizes	<img> <link> <source>	Specifies the size of the linked resource
span	<col> <colgroup>	Specifies the number of columns to span
spellcheck	Global Attributes	Specifies whether the element is to have its spelling and grammar checked or not
src	<audio> <embed> <iframe> <img> <input> <script> <source> <track> <video>	Specifies the URL of the media file
srcdoc	<iframe>	Specifies the HTML content of the page to show in the <iframe>
srclang	<track>	Specifies the language of the track text data (required if kind="subtitles")
srcset	<img> <source>	Specifies the URL of the image to use in different situations
start		Specifies the start value of an ordered list
step	<input>	Specifies the legal number intervals for an input field
style	Global Attributes	Specifies an inline CSS style for an element
tabindex	Global Attributes	Specifies the tabbing order of an element
target	<a> <area> <base> <form>	Specifies the target for where to open the linked document or where to submit the form

Attribute	Belongs to	Description
title	Global Attributes	Specifies extra information about an element

translate	Global Attributes	Specifies whether the content of an element should be translated or not
type	<a> <button> <embed> <input> <link> <menu> <object> <script> <source> <style>	Specifies the type of element
usemap	<img> <object>	Specifies an image as a client-side image map
value	<button> <input> <option> <meter> <progress> <param>	Specifies the value of the element
width	<canvas> <embed> <iframe> <img> <input> <object> <video>	Specifies the width of the element
wrap	<textarea>	Specifies how the text in a text area is to be wrapped when submitted in a form

Appendix – III
HTML Events

Global Event Attributes

- HTML has the ability to let events trigger actions in a browser, like starting a JavaScript when a user clicks on an element.
- To learn more about programming events, please visit our JavaScript tutorial.
- Below are the global event attributes that can be added to HTML elements to define event actions.

Window Event Attributes

Events triggered for the window object (applies to the <body> tag):

Event	Value	Description
onafterprint	script	Script to be run after the document is printed
onbeforeprint	script	Script to be run before the document is printed
onbeforeunload	script	Script to be run when the document is about to be unloaded
onerror	script	Script to be run when an error occurs
onhashchange	script	Script to be run when there has been changes to the anchor part of the a URL
onload	script	Fires after the page is finished loading
onmessage	script	Script to be run when the message is triggered
onoffline	script	Script to be run when the browser starts to work offline
ononline	script	Script to be run when the browser starts to work online
onpagehide	script	Script to be run when a user navigates away from a page
onpageshow	script	Script to be run when a user navigates to a page
onpopstate	script	Script to be run when the window's history changes
onresize	script	Fires when the browser window is resized
onstorage	script	Script to be run when a Web Storage area is updated
onunload	script	Fires once a page has unloaded (or the browser window has been closed)

Form Events

Events triggered by actions inside a HTML form (applies to almost all HTML elements, but is most used in form elements):

Event	Value	Description
onblur	script	Fires the moment that the element loses focus
onchange	script	Fires the moment when the value of the element is changed
oncontextmenu	script	Script to be run when a context menu is triggered
onfocus	script	Fires the moment when the element gets focus
oninput	script	Script to be run when an element gets user input
oninvalid	script	Script to be run when an element is invalid
onreset	script	Fires when the Reset button in a form is clicked
onsearch	script	Fires when the user writes something in a search field (for <input="search">)
onselect	script	Fires after some text has been selected in an element
onsubmit	script	Fires when a form is submitted

Keyboard Events

Event	Value	Description
onkeydown	script	Fires when a user is pressing a key
onkeypress	script	Fires when a user presses a key
onkeyup	script	Fires when a user releases a key

Mouse Events

Event	Value	Description
onclick	script	Fires on a mouse click on the element
ondblclick	script	Fires on a mouse double-click on the element
onmousedown	script	Fires when a mouse button is pressed down on an element
onmousemove	script	Fires when the mouse pointer is moving while it is over an element

onmouseout	script	Fires when the mouse pointer moves out of an element
onmouseover	script	Fires when the mouse pointer moves over an element
onmouseup	script	Fires when a mouse button is released over an element
onmousewheel	script	Deprecated. Use the onwheel attribute instead
onwheel	script	Fires when the mouse wheel rolls up or down over an element

Drag Events

Event	Value	Description
ondrag	script	Script to be run when an element is dragged
ondragend	script	Script to be run at the end of a drag operation
ondragenter	script	Script to be run when an element has been dragged to a valid drop target
ondragleave	script	Script to be run when an element leaves a valid drop target
ondragover	script	Script to be run when an element is being dragged over a valid drop target
ondragstart	script	Script to be run at the start of a drag operation
ondrop	script	Script to be run when dragged element is being dropped
onscroll	script	Script to be run when an element's scrollbar is being scrolled

Clipboard Events

Event	Value	Description
oncopy	script	Fires when the user copies the content of an element
oncut	script	Fires when the user cuts the content of an element
onpaste	script	Fires when the user pastes some content in an element

Media Events

Events triggered by medias like videos, images and audio (applies to all HTML elements, but is most common in media elements, like <audio>, <embed>, <img>, <object>, and <video>).

Event	Value	Description
onabort	script	Script to be run on abort
oncanplay	script	Script to be run when a file is ready to start playing (when it has buffered enough to begin)
oncanplaythrough	script	Script to be run when a file can be played all the way to the end without pausing for buffering
oncuechange	script	Script to be run when the cue changes in a <track> element
ondurationchange	script	Script to be run when the length of the media changes
onemptied	script	Script to be run when something bad happens and the file is suddenly unavailable (like unexpectedly disconnects)
onended	script	Script to be run when the media has reach the end (a useful event for messages like "thanks for listening")
onerror	script	Script to be run when an error occurs when the file is being loaded
onloadeddata	script	Script to be run when media data is loaded
onloadedmetadata	script	Script to be run when meta data (like dimensions and duration) are loaded
onloadstart	script	Script to be run just as the file begins to load before anything is actually loaded
onpause	script	Script to be run when the media is paused either by the user or programmatically
onplay	script	Script to be run when the media is ready to start playing
onplaying	script	Script to be run when the media actually has started playing
onprogress	script	Script to be run when the browser is in the process of getting the media data

onratechange	script	Script to be run each time the playback rate changes (like when a user switches to a slow motion or fast forward mode)
onseeked	script	Script to be run when the seeking attribute is set to false indicating that seeking has ended
onseeking	script	Script to be run when the seeking attribute is set to true indicating that seeking is active
onstalled	script	Script to be run when the browser is unable to fetch the media data for whatever reason
onsuspend	script	Script to be run when fetching the media data is stopped before it is completely loaded for whatever reason
ontimeupdate	script	Script to be run when the playing position has changed (like when the user fast forwards to a different point in the media)
onvolumechange	script	Script to be run each time the volume is changed which (includes setting the volume to "mute")
onwaiting	script	Script to be run when the media has paused but is expected to resume (like when the media pauses to buffer more data)

Misc Events

Event	Value	Description
ontoggle	script	Fires when the user opens or closes the <details> element

AGM RURAL COLLEGE OF ENGINEERING AND TECHNOLOGY, VARUR

Department of Computer Science & Engineering

Web Technology Laboratory

Laboratory Manual



Prepared by, Mr. Chandrakant Hatti

BCSL504

Web Technology Laboratory

Laboratory Manual

Course Details

Course Code:	BCSL504	*CIE Marks:*	50
Number of Lecture Hours/Week:	02 Hours Laboratory	*SEE Marks:*	50
RBT Levels:	L1, L2, L3	*Exam Hours:*	03

Course Outcomes: On the completion of this laboratory course, the students will be able to:

1. Design the experiment for the given problem using HTML, Javascript and CSS
2. Develop the solution for the given real-world problem using jQuery, Ajax and PHP
3. Analyze the results and produce substantial written documentation